TON DUC THANG UNIVERSITY

[Faculty name – fill in]

Thiết kế và Đánh giá hiệu năng hệ thống Microservices xử lý giao dịch song song quy mô lớn trong kịch bản Flash-sale

Capstone Project Report

Students
Vũ Đình Kiệt (523V0011)
Vũ Tấn Minh (523V0012)
Class 23K50201

Supervisor
Mai Văn Mạnh

October 23, 2026

Contents

1 Introduction **5**

2 Problem Statement **6**

3 Objectives **7**

4 Scope **8**

5 Limitations **9**

6 System Requirements **10**
6.1 Functional Requirements 10
6.2 Non-Functional Requirements 11

7 Technology Selection **13**

8 Development Environment **15**
8.1 Local infrastructure (Docker Compose) 15
8.2 Toolchain 15

9 System Analysis and Design **18**
9.1 Entity-Relationship Diagram 18
9.2 Class Diagram 20
9.3 API Contract 22
9.4 Event Contract 22
9.5 Order State Machine 23
9.6 Sequence Diagrams 24
9.7 Idempotency, Ordering and Ownership 25
9.8 Design Rationale 28
9.9 Supported Failure Model 29

10 Implementation **32**
10.1 Baseline Configurations — C0 and C1 (Week 3) 32
10.1.1 Reset and seed 32

10.1.2 C0 – the naive baseline (a controlled failure demonstration) 32
10.1.3 C1 – the PostgreSQL atomic update 33
10.1.4 Correctness criteria 34
10.1.5 JMeter smoke test 35
10.2 Order Service 35
10.2.1 API design 36
10.2.2 Idempotency strategy 36
10.2.3 State management 37
10.2.4 Test cases 38
10.2.5 Event Publication Reliability (Week 6) – the dual-write problem and the Transactional Outbox 38
10.3 Inventory Service 42
10.3.1 Scaffolding 42
10.3.2 Redis key and data design 42
10.3.3 Lua reservation logic 42
10.3.4 Inventory warm-up and readiness 43
10.3.5 Inventory invariants under concurrency 43
10.3.6 OrderPlaced handling and the Redis/PostgreSQL boundary . 44
10.4 Closing the Loop – Stock Results and Order State (Week 8) 45
10.4.1 Inventory Service's own Outbox – a harder dual-write problem 45
10.4.2 Order Service: consuming the result 46
10.4.3 Automated correctness validation 46
10.4.4 The main workflow (Step 8.4) 48
10.5 Reliability and Idempotency Layer (Week 9) 51
10.5.1 Message delivery semantics 51
10.5.2 Idempotent consumer 51
10.5.3 Acknowledgement strategy, proven with a real crash (Step 9.3) 53
10.5.4 Duplicate handling 54
10.5.5 Concurrent duplicates and out-of-order completion (TP-M02) 55
10.6 Failure Handling and Recovery (Week 10) 57
10.6.1 Classify, then retry a bounded number of times 57
10.6.2 A dead-letter queue per service, and replay 58
10.6.3 Reconciliation: stuck orders, a bound and an alert 60
10.6.4 Supported and unsupported failure modes 62
10.7 Process Worker and Observability (Week 11) 64
10.7.1 The Process Worker, and what it settled about the state machine 64
10.7.2 Correlation IDs across three services (Step 11.2) 68
10.7.3 Structured logging and metrics (Step 11.3) 70

10.8 Load Shedding 76
10.8.1 Overload, drain and recovery (preliminary, local) 80
10.9 Deployment on AWS EC2 81
10.10 Verification Summary 84
10.11 External Review after Week 12 85

11 Experimental Design **87**

12 Results **88**
12.1 Measurement Method 88
12.2 Correctness by Configuration 89
12.3 Latency by Configuration 89
12.4 Throughput, Rejection and Errors 91
12.5 Overload, Backlog and Recovery 91
12.6 Resource Use 92
12.7 Figures 93

13 Discussion **95**

14 Conclusion **96**

References **97**

A Appendices **98**

1

Introduction

Flash-sale workloads send a short burst of concurrent orders toward a small amount of inventory. A backend must preserve stock correctness while processing requests quickly, and must control work when the incoming rate exceeds available capacity.

This project compares a synchronous PostgreSQL inventory baseline with an asynchronous design using RabbitMQ and an atomic Redis Lua reservation. It evaluates correctness, request and completion latency, throughput, overload response, and the effect of running multiple consumers on one host. The research question, boundaries, and experiment controls are recorded in `docs/00-scope-lock.md` and `docs/experiment-protocol.md`.

2

Problem Statement

With finite stock, a read-check-write sequence can let concurrent requests reserve the same remaining unit. A conditional atomic update can prevent that race, but contention may affect latency and throughput. In an asynchronous design, separate services also have to handle partial failures, redelivered messages, and traffic above capacity without losing track of accepted orders.

This project asks how C0 (a deliberately unsafe demonstration), C1 (synchronous PostgreSQL atomic update), and C2 (RabbitMQ with Redis Lua reservation) behave under matched workloads. C3 adds load shedding; C4 compares 1, 2, and 4 Inventory Service replicas on the same EC2 host. C0 is only a race-condition demonstration; C1 is the correctness and performance baseline. The study measures the trade-offs rather than assuming that a relational database or a microservices architecture is inherently faster or safer.

3

Objectives

1. Compare C0 and C1 with the asynchronous C2 design under equivalent conditions; evaluate C3 overload controls and C4 same-host replica counts.
2. Check the inventory rules in `docs/inventory-invariants.md`, including no negative stock, no oversell, one reservation per order, and no duplicate side effects on redelivery.
3. Measure request acceptance and end-to-end completion separately, alongside latency percentiles, throughput, rejection, queue, resource, retry, and DLQ measures.
4. Determine whether accepted orders reach a terminal state or are surfaced by reconciliation, and limit conclusions to the tested hardware, workloads, and failure scenarios.

4

Scope

See `docs/00-scope-lock.md` for the research question and scope. The system includes Order Service, Inventory Service, a deterministic Process Worker, PostgreSQL schemas owned by each service, Redis Lua stock reservation, RabbitMQ, and the Nginx, application and queue load-shedding controls of C3. The backend target is one AWS EC2 host; JMeter runs on a separate machine. C4 varies Inventory Service replicas as 1, 2, and 4 processes or containers on that same host.

The study excludes a frontend, authentication and authorization, real payment processing, payment-failure compensation, multi-node scaling, clusters, and multi-region deployment. It does not claim that one-host replication represents multi-node horizontal scaling.

5

Limitations

The results have the following limits:

- PostgreSQL, Redis, and RabbitMQ each run as a single instance. The project does not evaluate cluster failover, infrastructure node loss, or multi-region network effects.
- The Process Worker uses a deterministic delay and successful result; the study does not characterize real downstream failure rates or compensation.
- Replica experiments use processes or containers on one EC2 host. Shared CPU, memory, and I/O contention limit how far their results can be generalized.
- Load-shedding thresholds and the reservation-attempt target are specific to the recorded hardware and workload; they are not universal capacity guarantees.
- Authentication and authorization are out of scope. The order-status API therefore must not be presented as production-secure, and a GUID is not a substitute for an authorization check.

6 System Requirements

Requirements below follow the teacher brief (`docs/teacher-brief.md`, §4) item by item. They state what the system must do; whether each is implemented and verified is reported in the chapter for the week that builds it, not assumed here.

6.1 Functional Requirements

1. **FR-1 Order intake.** Accept and persist an order for one product and return its order ID as soon as the request is accepted, in state `PendingStock`, without waiting for inventory.
2. **FR-2 Status polling.** Let the client poll an order's current state until it reaches a terminal state or times out.
3. **FR-3 Atomic reservation.** Check and reserve inventory atomically (Redis Lua in C2–C4; conditional atomic update in PostgreSQL for C1), so concurrent requests can never reserve more than the available stock.
4. **FR-4 Status from reservation result.** Move the order to `Confirmed` or `Rejected` from the `StockReserved` or `StockRejected` event.
5. **FR-5 Downstream processing.** A Process Worker consumes `StockReserved`, simulates downstream work with a deterministic delay and a successful result, and uses the order ID as its idempotency key.
6. **FR-6 Messaging.** Publish and consume events through RabbitMQ, with publisher confirms.
7. **FR-7 Transactional Outbox.** Every producer writes its business change and its outgoing event in one local transaction; a publisher retries until the broker confirms.
8. **FR-8 Inbox.** Every consumer records each message ID under a unique constraint, in the same transaction as its business change, so a redelivered message has no second effect.

9. **FR-9 Retry and DLQ.** Retry transient failures a bounded number of times; move poison or exhausted messages to a dead-letter queue.
10. **FR-10 Reconciliation.** A periodic job finds orders stuck in a non-terminal state too long. It is a safety net, not a replacement for the Outbox, retry or DLQ.
11. **FR-11 Warm-up gate.** Load inventory into Redis before the sale and open the sale only after warm-up is complete and confirmed.
12. **FR-12 Stage timestamps and history.** Record the state and a timestamp at each processing stage, including a state-transition history.
13. **FR-13 Client and message deduplication.** Reject duplicate client submissions (Idempotency-Key) and ignore duplicate or invalid events without repeating a state transition.
14. **FR-14 Load shedding.** When configured limits are reached (Nginx rate limit, application concurrency limit, bounded RabbitMQ queue), reject new work early with a suitable status code, before an order is created. Never return "accepted" for work the system cannot guarantee to continue. Evaluated in C3.

6.2 Non-Functional Requirements

- **Consistency.** In every tested workload and failure scenario: inventory never goes negative; successful reservations never exceed the initial inventory; an order is reserved at most once; a duplicate message causes no duplicate side effect. Checked after each run by the four invariants in `docs/inventory-invariants.md`; a failed or inconclusive run is reported, never counted as a pass.
- **Performance.** Evaluate at several load levels. Every experiment records the EC2 configuration, virtual users and request rate, test duration, number of products, initial inventory, number of consumer replicas, payload size, and the PostgreSQL, Redis and RabbitMQ configuration. The initial target is about 2,000 reservation attempts per second on the chosen hardware; this is an experimental target, not a hardware-independent guarantee. API throughput and business-completion throughput are reported separately.
- **Resilience.** Under overload, reject part of the load in a controlled way instead of accepting without limit. After load drops, the backlog must drain and the system must return to a stable state without a full restart.

- **Observability.** Every order can be traced by correlation ID, order ID, message ID, current status and state-transition history, plus four timestamps: request acceptance, event publication, stock processing and completion.
- **Liveness.** Every accepted order reaches a terminal state or is surfaced by reconciliation; order counts by state must reconcile with the number of accepted requests.
- **Replication scope.** Compare 1, 2 and 4 Inventory consumer replicas on the same EC2 host; do not claim multi-node scalability from these runs.
- **Reproducibility.** Reset, seed and warm up before each run; repeat each configuration 3–5 times; report percentiles, median and run-to-run variation, not only averages.

7

Technology Selection

ABP Framework 10.6 on .NET 10

Required by the brief. Used for module wiring, EF Core integration and hosting, from the `app-nolayers` template to keep each service's codebase proportional to its size. ABP's own RabbitMQ event bus (`Volo.Abp.EventBus.RabbitMQ`) is *not* used: messaging goes through `RabbitMQ.Client` 7.1.2 directly, because the design needs per-message publisher confirms and plain (non-enveloped) JSON messages per event, which the implementation chapters explain.

PostgreSQL 16

The durable, transactional side (orders, Outbox, Inbox). The Transactional Outbox needs the business write and the event write to commit in one ACID transaction, so a relational engine is a prerequisite. One instance with a schema and a database account per service (schema-per-service, not physical isolation). It also hosts the C1 baseline's conditional atomic update.

Redis Stack with Lua

Holds available inventory and the reservation-level duplicate set. `scripts/redis/rese` checks idempotency, checks stock, decrements and records the result atomically, in one script call. Accessed through `StackExchange.Redis`. RedisInsight (bundled with Redis Stack) is for inspection only.

RabbitMQ vs. Kafka

RabbitMQ fits this workload: routed queues per consumer, publisher confirms, bounded retry, dead-lettering and bounded queues with an overflow policy for load shedding. Historical log replay, Kafka's main advantage, is not part of the research scope.

Nginx

Entry point and first load-shedding layer (`limit_req` rate limiting). Sufficient for a single-host deployment without a full API gateway.

Serilog, prometheus-net, Prometheus, Grafana

Structured JSON logs carrying correlation, order and message IDs (Serilog),

service metrics exported by `prometheus-net`, scraped by Prometheus and viewed in Grafana.

Docker Compose

Runs PostgreSQL, Redis, RabbitMQ, Nginx and the monitoring stack identically on a developer machine and the EC2 host.

Terraform and AWS EC2

The brief requires the system under test on one EC2 instance. Terraform (`infra/terraform/`) creates that host and a separate JMeter host reproducibly and removes them again (Section 10.9).

Apache JMeter

Simulates concurrent clients through the order and status APIs. It runs on a separate machine so it does not compete with the system under test.

8 Development Environment

8.1 Local infrastructure (Docker Compose)

`infra/docker-compose.yml` defines the local data, broker, proxy, and monitoring containers. From the repository root, start them with `docker compose -f infra/docker-compose.yml up -d`:

Service	Image	Host ports
PostgreSQL	`postgres:16`	5432
Redis Stack	`redis/redis-stack:latest`	6379 (data), 8001 (RedisInsight UI)
RabbitMQ	`rabbitmq:3-management`	5672 (AMQP), 15672 (management UI)
Nginx	`nginx:latest`	8080 (host) → 80 (container)
Prometheus	`prom/prometheus:latest`	9090
Grafana	`grafana/grafana:latest`	3000

The .NET services currently run on the host; Prometheus reaches them through `host.docker.internal`. PostgreSQL's `flashsale` account is for local bootstrap only. Order Service and Inventory Service use separate local roles created by `infra/initdb/01-create-service-roles.sql`. These development passwords are not deployment secrets. Mutable image tags must be pinned before official experiments.

8.2 Toolchain

`setup.ps1` contains Windows onboarding defaults, not a record that every tool was installed or verified on the benchmark host. Record the actual .NET SDK, Docker images, Terraform version, JMeter version, and AWS instance configuration in each experiment's run manifest.

Tool	Use
Git	Source control
.NET SDK	Build and run the services
Docker Compose	Run local dependencies
Terraform	Create and remove the EC2 backend and JMeter hosts
Apache JMeter	Generate load from a separate machine

Pinned versions. The .NET SDK is pinned by `global.json` to 10.0.300 (any later 10.0.3xx patch is accepted; other versions fail fast). All projects target `net10.0` and use ABP 10.6.0, the LeptonX Lite theme 5.4.0 and `RabbitMQ.Client` 7.1.2. No package uses a floating version, so a clean build resolves the same dependencies on every machine.

Fresh-environment check. `scripts/verify-environment.ps1` starts a temporary PostgreSQL container and proves the setup from nothing. It checks the SDK against `global.json`, then does a clean build of the whole solution. It runs both services' migrations, each with its own database account, and applies the baseline schema. Finally it checks account isolation: each service can use its own schema, cannot read the other's, and only Order Service can use `public`. It also checks the schema against the ERD. On 2026-10-01 it passed from a fresh clone with 0 build errors and 0 warnings. With `-Mode Existing`, the same checks run on a copy of an existing database after the role script is applied twice.

Runtime check on the local stack (TP-E04). The checks above use temporary databases. A 2026-10-13 audit found that the project's own local Compose database had never had the role script applied: it had no service accounts, and every table was owned by the admin account. The documented existing-database procedure was re-run on a copy (49/49), then applied to the real database after a backup. `scripts/verify-runtime-accounts.ps1` then starts all three services with their checked-in settings against the local Compose stack and places three orders, which all complete. While the services run, every Order Service database session belongs to `order_service_user`, and every Inventory session to `inventory_service_user`. Each connection string sets an `Application Name`, so every session can be tied to its service. No service connects as the admin account, and the Process Worker has no database at all. With the services' own credentials, each cross-schema read is refused ("permission denied for schema"), and each own-schema read succeeds. The check passed 61/61, including the ERD comparison. A negative control that switches Inventory to the admin account fails three checks.

Run project commands from the repository root and resolve script paths relative to the script location. Do not rely on a machine-specific drive letter or an untracked setup checklist.

9 System Analysis and Design

9.1 Entity-Relationship Diagram

The data model below is taken from the EF Core models and migrations, the raw baseline SQL and the role/schema initialisation script, not drawn by hand. `docs/erd.md` holds the same diagrams plus a machine-readable manifest, and `scripts/verify-erd.ps1` compares that manifest with a live database. On 2026-10-01 a database built from scratch (role initialisation, then both services' migrations, then the baseline schema) passed all 29 checks.

Architecture of the data layer. One PostgreSQL instance holds one schema per service. Order Service writes `order_service`, plus ABP's module tables in `public`. Inventory Service writes only `inventory_service`. Each schema is owned by its own login role (`order_service_user`, `inventory_service_user`). Neither role can read the other's schema, and only Order Service's role can use `public`. The services share data only through RabbitMQ events. There are no foreign keys: links such as an Outbox row's `OrderId` are logical, and they are kept consistent by writing both rows in one local transaction. Inventory's `OrderId` refers to an order in another service's schema, so it could not have a foreign key anyway. In the asynchronous design, stock is held in Redis, not PostgreSQL: `inventory:{productId}` stores available units and `processed:{productId}` stores the ids of orders already reserved.



Figure 9.1. – `order_service` schema (Order Service). The Outbox row references its order logically through `OrderId`. The Inbox (`processed_messages`) is unique per broker `MessageId`.

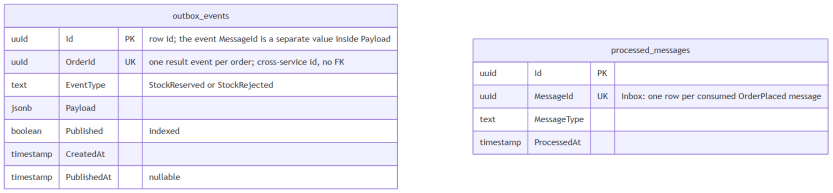


Figure 9.2. – `inventory_service` schema (Inventory Service). It has both an Outbox, unique per `OrderId` so one order yields at most one stock result event, and an Inbox, unique per `MessageId`.

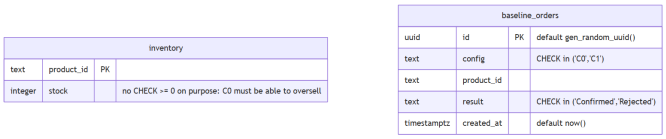


Figure 9.3. – C0/C1 baseline tables (raw SQL, not EF-migrated). `stock` deliberately has no non-negative check so that C0 can demonstrate overselling.

Unique constraints. Four unique indexes carry the system's idempotency guarantees: `orders.IdempotencyKey` (one order per client key), `processed_messages.MessageId` in each service schema (each broker message is applied at most once by each consumer), and `inventory_service.outbox_events.OrderId` (at most one stock result per order). Non-unique indexes support trace lookup (`orders.CorrelationId`) and Outbox polling (`Published`).

ABP and the `public` schema. Order Service is an ABP application. Its first migration creates 37 ABP module tables (`Abp*`, `OpenIddict*`) and its EF migration

history in `public`, all owned by `order_service_user`. They stay in `public` because forcing a default schema breaks some ABP modules at runtime. Inventory Service configures no ABP modules. Its connection string sets the search path to `inventory_service`, so its migration history is stored there. No business or experiment data lives in `public`.

Ownership defects found by this check. Two were found and fixed. First, on a fresh database the baseline tables were owned by the `flashsale` admin account that creates them, so `order_service_user` had no privileges on them and the C0/C1 runs would have failed. The baseline schema script now hands them to `order_service_user`. Second, on a database created before the per-service accounts existed, both services shared one migration-history table in `public`. Once Inventory Service connected with its own search path, it re-ran its migrations and failed with "relation already exists". The role script now moves Inventory's history rows into `inventory_service`. Both the fresh and the existing-database paths are checked by `scripts/verify-environment.ps1`.

Since Week 11, the Order Service schema also holds `order_status_history`, the per-order transition history required by the six-state model (Section 9.5). Each row records the from-state, the to-state, the triggering `MessageId`, the time, and whether the step was implied. An identity column, `Seq`, gives the order in which the rows were written, and every read of the history uses it.

9.2 Class Diagram

The domain and messaging classes below are taken from the source (`Entities/` in Order and Inventory Service and the shared `FlashSale.EventContracts` library; `docs/class-diagram.md`). `scripts/verify-design-alignment.ps1` checks that every class and property in the diagrams exists in code with the same type, and that no public property is missing. Inventory Service has its own `OutboxEvent` and `ProcessedMessage` classes in its own schema; the two services share no entity classes, only the event contracts.

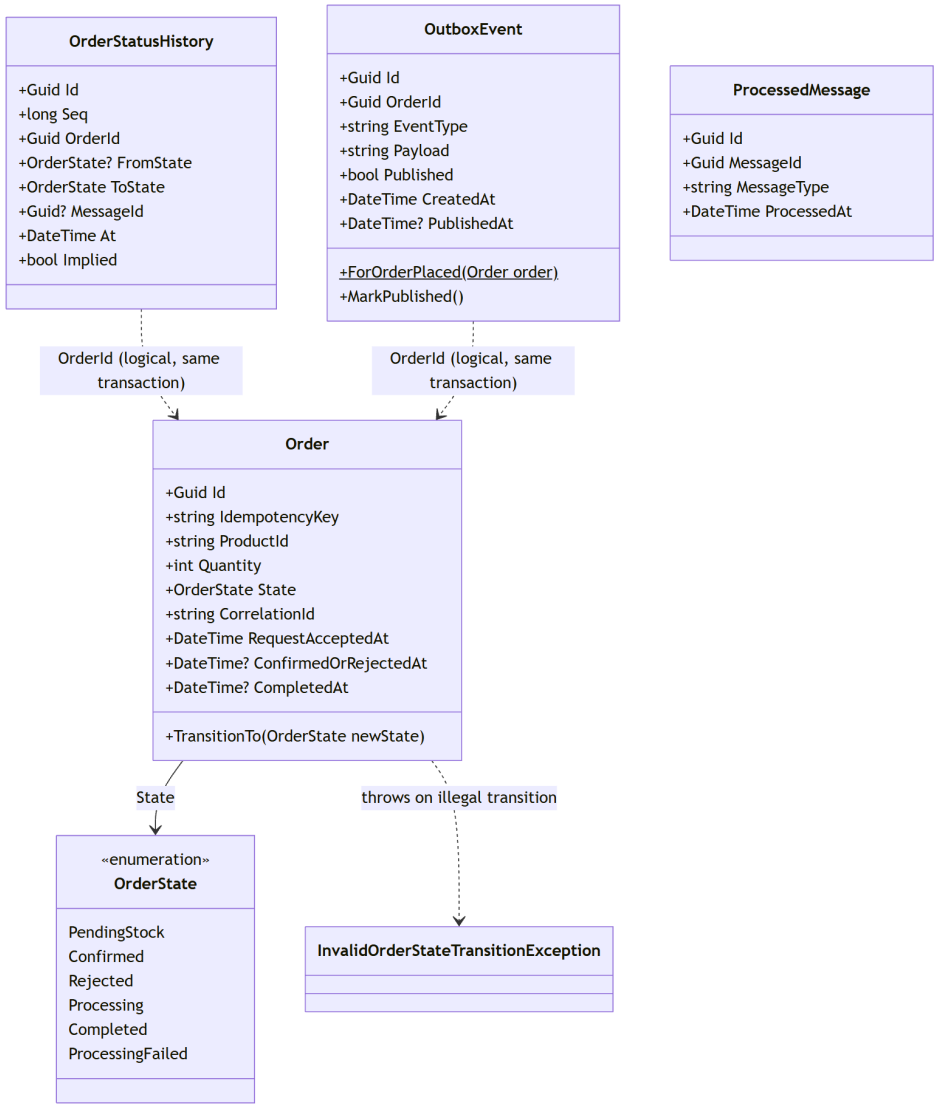


Figure 9.4. – Order Service domain classes. `Order.TransitionTo` enforces the state machine; the Outbox row refers to its order only logically.

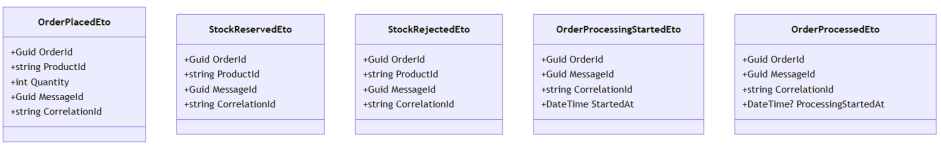


Figure 9.5. – Event contracts shared by all services. Every event carries a `MessageId` (Inbox key) and a `CorrelationId` (trace key).

Week 11 added the six-state model to the code: `OrderState.Processing`, `OrderState.ProcessingFailed`, the `OrderProcessingStartedEto` event and the `OrderStatusHistory` entity. All of them are drawn above.

9.3 API Contract

ABP generates the OpenAPI document from the running controllers; it is exported to `docs/api-contract-v1.json`. Since 2026-10-01 each order endpoint declares its outcomes with `[ProducesResponseType]`, so the export lists every status code the code can return. The alignment check compares the committed file with the live export of the same build.

Endpoint	Status	Meaning
`POST /api/orders`	201	New order accepted in `PendingStock`; body is the order with its id
	200	Same `Idempotency-Key` and same body as an earlier request: the original order
	400	`Idempotency-Key` missing, or invalid body (quantity $\neq 1$, blank product)
	409	Key already used with a different body
`GET /api/orders/{id}`	200	Current state and the stage timestamps
	404	Unknown order id
`POST /api/c1/orders`	200	C1 baseline decision: `Confirmed` with remaining stock, or `Rejected`

Request validation (quantity must equal 1, non-blank product and key) was added in Week 5 and is verified by case TP-A02. Since Week 12 a new request is also rejected before order creation when admission capacity is exhausted (503, case TP-L01). The remaining paths in the export are ABP module scaffolding (identity, account, permissions, multi-tenancy). The scope lock excludes them, and they play no part in the order workflow.

9.4 Event Contract

Full contract: `docs/event-contract.md`; the classes live in `FlashSale.EventContracts`. All events go through one direct exchange, `flashsale.order.exchange`, with the event name as routing key.

Event	Publisher	Consumer(s)
`OrderPlacedEto`	Order Service (Outbox)	Inventory Service
`StockReservedEto`	Inventory Service (Outbox)	Order Service, Process Worke
`StockRejectedEto`	Inventory Service (Outbox)	Order Service
`OrderProcessingStartedEto`	Process Worker	Order Service
`OrderProcessedEto`	Process Worker	Order Service

Each event carries `MessageId`, regenerated at every hop because each consumer keeps its own Inbox, and `CorrelationId`, copied unchanged from the inbound request so one value traces the whole journey. The Process Worker sets `MessageId` equal to `OrderId`, because it has no database of its own; a replay then carries the same `MessageId`, and the Inbox drops it. `StockRejectedEto` has no `Reason` field, because insufficient stock is the only rejection cause in scope.

9.5 Order State Machine

See `docs/order-state-machine.md` for the full diagram and transition table. The adopted model (decided 2026-10-01) uses the teacher's six named states: `PendingStock`, `Confirmed`, `Rejected`, `Processing`, `Completed` and `Processing-Failed`. `Processing` is reachable through a dedicated `OrderProcessingStarted` event published by the Process Worker when it begins work, followed by `OrderProcessed` for `Processing` → `Completed`. `ProcessingFailed` is declared but unreachable by design: the scope lock gives the Process Worker a deterministic, successful outcome only, so no in-scope event produces it (the success-only interpretation). Reordered or duplicated Worker events are absorbed by explicit rules (stale events are no-ops; an early `OrderProcessed` applies an implied `Processing` step in the same transaction), and every transition is appended to an order status history. Since Week 11 the code implements exactly this model, and the alignment check compares the transition table with `Order.LegalTransitions`. The implementation and its verification are described in Section 10.7.

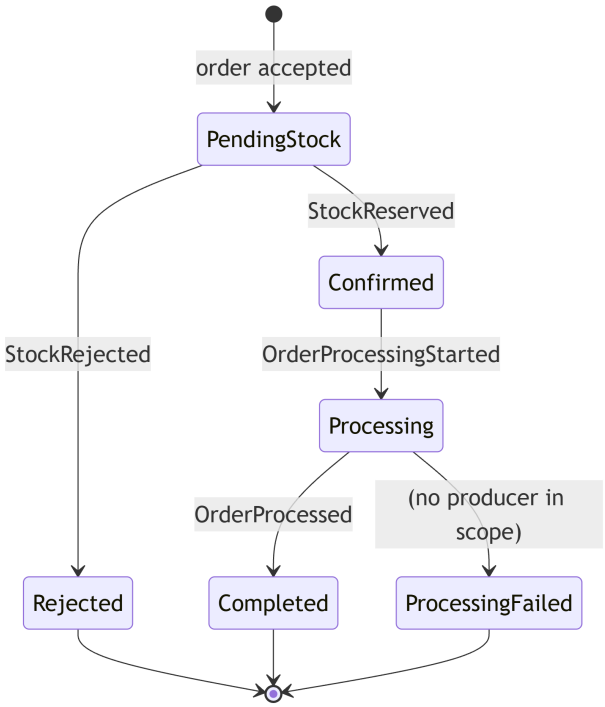


Figure 9.6. – Adopted six-state order model. `ProcessingFailed` has no producer in scope.

9.6 Sequence Diagrams

The full diagrams are in `docs/sequence-diagrams.md`, and the numbered steps are described in `docs/workflow.md` (Section 10.4.4). The diagrams show the code as built in Week 8, and every arrow corresponds to a call in the source:

- both Outboxes, with their 2-second publishers;
- the routed-confirmation check of each event's required queues;
- the Inbox check before each consumer's business step;
- the Lua reservation, including the `NOT_OPEN` warm-up gate;
- the concurrent delivery of `StockReserved` to Order Service and to the Process Worker;
- status polling.

Since Week 12, clients post and poll through Nginx, which applies the C3 rate limit (Section 10.8). The `Processing` hop of the six-state model is drawn in a separate diagram. Retry and DLQ branches are listed per step in Section 10.4.4 rather than drawn.

The diagrams are checked against the implementation by test case TP-W04 (`scripts/verify-workflow.ps1`), in two ways:

- 38 static checks compare each arrow with the code and the `appsettings.json` files.
- A live run traces real orders through all three services and checks their database rows, timestamps, Redis keys and logs against the step order.

9.7 Idempotency, Ordering and Ownership

The full definitions, each tied to the code that implements it, are in `docs/design-decisions.md` and are checked by `scripts/verify-design-decisions.ps1`.

Transport versus business idempotency. Two *transport* keys absorb retries of the same request or message. The client's `Idempotency-Key` is unique per order, and a reused key with a different body returns 409. The broker's `MessageId` is unique per consumer Inbox and is written in the same transaction as the business change. The *business* key is `OrderId`. Redis's per-product set of reserved order ids and the unique `OrderId` on Inventory's Outbox guarantee one deduction and one result per order even when the same fact arrives as a new message, for example the reconciliation job re-publishing `OrderPlaced`.

Ordering and replay. The design does not rely on message order. Duplicates and events for a state already reached are acknowledged as no-ops. A conflicting result goes to the service's dead-letter queue. An event that arrives before its prerequisite is retried, never dropped. Replaying any message leaves stock, state and Outbox rows unchanged.

Schema ownership. Each schema has one owner account (`order_service`, plus ABP's `public` tables, belongs to Order Service; `inventory_service` belongs to Inventory Service). Redis inventory keys belong to Inventory Service. Services never read each other's data; facts cross service boundaries only as events.

Redis/PostgreSQL boundary. Redis decides the reservation atomically; PostgreSQL then records the result (Outbox row and Inbox row) in one transaction, and only then is the message acknowledged. If PostgreSQL fails after Redis has reserved, the message is redelivered. The Lua script answers `DUPLICATE` without a second deduction, and the missing result is written.

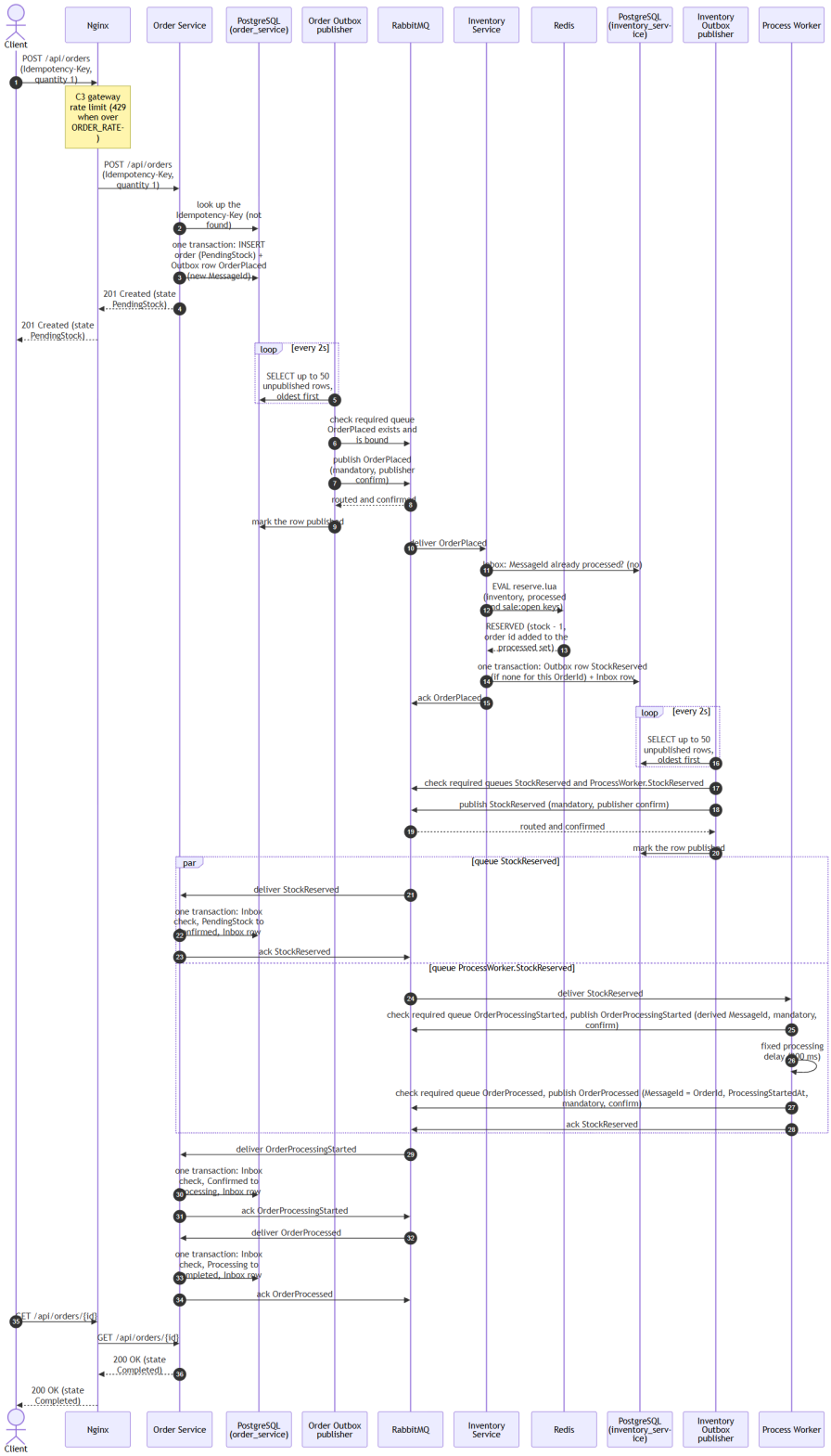


Figure 9.7. – Happy path as built (Week 8): order accepted, stock reserved, result applied, processed, polled as `Completed`. The step numbers match Table 10.1.

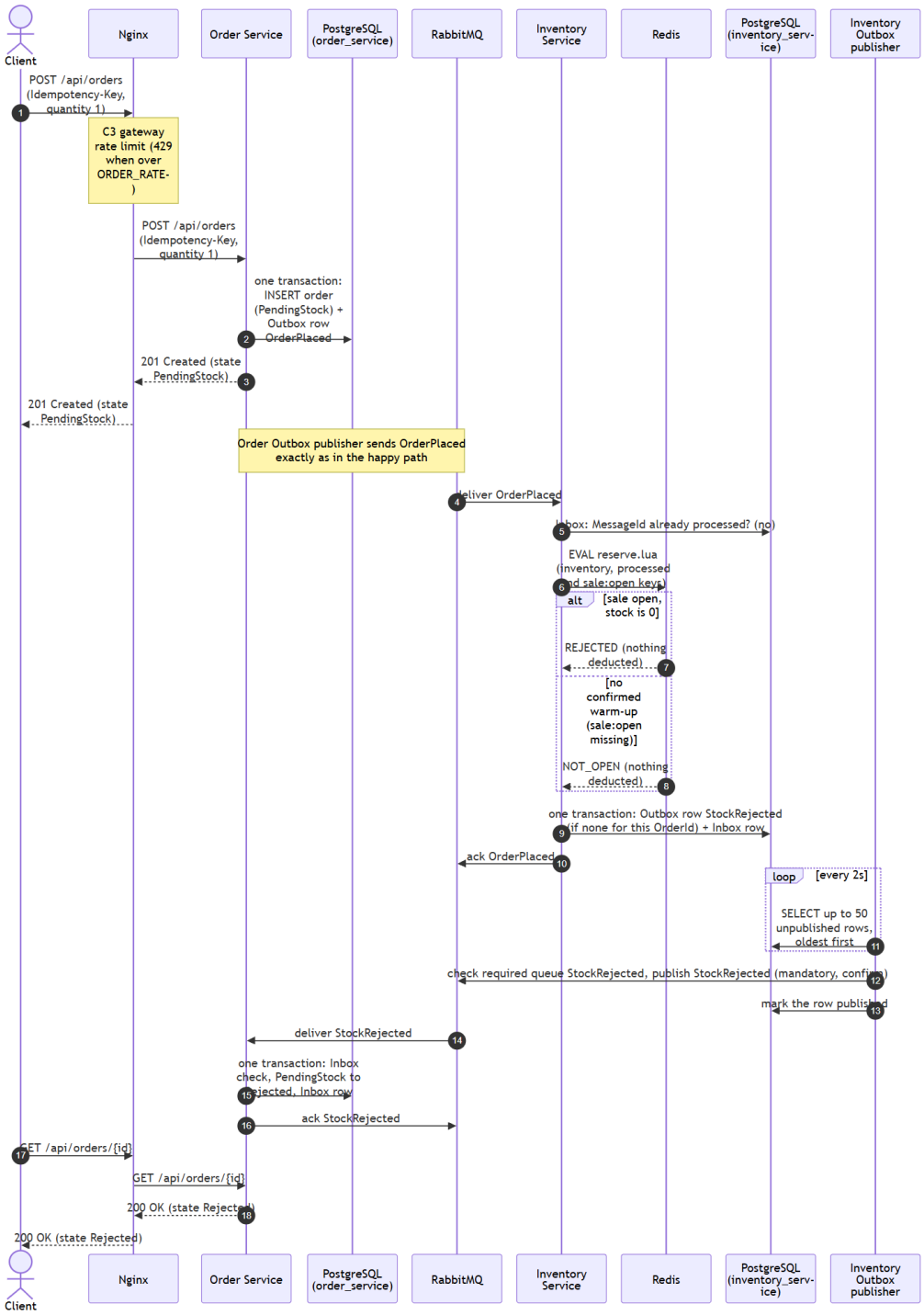


Figure 9.8. – Rejection path: the same as the happy path up to the Lua script. `REJECTED` (sold out) and `NOT_OPEN` (no confirmed warm-up) both lead to `StockRejected` and the terminal state `Rejected`. The Process Worker is not involved.

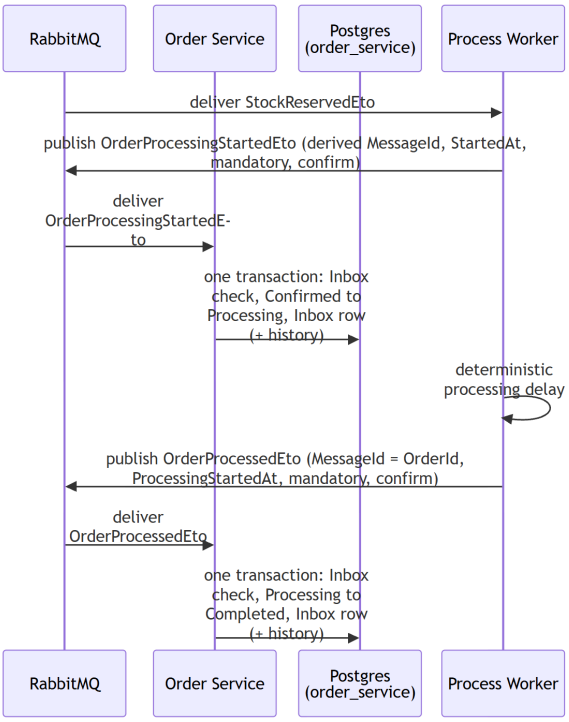


Figure 9.9. – The `Processing` hop of the six-state model (Week 11): the Worker's start event moves the order to `Processing`, and its completion moves it to `Completed`.

9.8 Design Rationale

The project compares a synchronous PostgreSQL conditional update (C1) with an asynchronous, event-driven design (C2–C4) under the same correctness invariants. Each mechanism in the asynchronous design was chosen for one reason:

- **Accept first, decide later (HTTP 201 in `PendingStock`).** It separates the request acceptance rate from the processing rate, so API latency and completion latency are measured separately, as the brief requires.
- **Redis Lua for the stock decision.** It makes the idempotency check, the stock check and the decrement one atomic step in memory, instead of a row lock that every buyer of a hot product waits on in C1.
- **Transactional Outbox on both producers.** It removes the dual-write problem: an order (or a stock result) and its event commit together or not at all, and a publisher retries until the broker confirms.
- **Inbox per consumer.** RabbitMQ delivers at least once; a unique `MessageId` written with the business change makes every effect happen at most once.

- **Direct exchange, one queue per consumer.** Order Service and the Process Worker each get their own copy of `StockReserved`; sharing a queue would split the messages between them.
- **Schema-per-service on one PostgreSQL instance.** It gives ownership and isolation by account at the cost of a single shared server, which the brief accepts as a limitation (not physical isolation).
- **Success-only Process Worker.** It keeps the downstream stage deterministic, so latency variation in the results comes from the architecture rather than from simulated failures. That is why `ProcessingFailed` has no producer.

The cost of these choices is complexity: three services, two Outboxes, two Inboxes, a broker and a cache, compared with one SQL statement in C1. The Experimental Design and Results chapters quantify what this buys. Replica results (C4) are limited to processes on one host and do not show multi-node scaling.

9.9 Supported Failure Model

The table lists the failures the design handles, how it handles them, and where that handling is verified. “Designed” means the mechanism exists in code but has no saved verification yet; it is not a claim that it works.

Failure	Handling	Verification
Client retries the same request (also concurrently)	`Idempotency-Key`: same body returns the original order, different body returns 409; a lost unique-index race is answered as a replay	Verified: TP-A03–A05 (W05)
Broker redelivers a message	Inbox (unique `MessageId`) in the same transaction; ack after commit	Unit tests pass (TP-E03); live: TP-M01 (W09)
Same order arrives as a new message	Redis `DUPLICATE`; unique `OrderId` on Inventory Outbox; guarded state machine	Verified: TP-M02 (W09)
Crash after commit, before ack	Redelivery hits the Inbox and is acked	Verified: TP-M01 (W09)
Broker unreachable while publishing	Outbox row stays unpublished; publisher retries; consumers reconnect every 5 s	Verified: TP-O01, TP-O02 (W06)
PostgreSQL fails after Redis reserved	Not acked: retried, then dead-lettered; a retry or replay gets Lua `DUPLICATE` and writes the missing result once	Verified: TP-I03 (W07)
Transient handler error	Bounded retry (4 attempts: 1 s, 2 s, 4 s), then the service's DLQ	Verified: TP-F01 (W10)
Poison message or state conflict	Straight to the service's DLQ, no retry	Verified: TP-F01 (W10)
Order stuck in `PendingStock`, `Confirmed` or `Processing`	Reconciliation job re-publishes `OrderPlaced` after a timeout, at most 3 times, then an alert; Inventory re-sends a stored result, which also reaches the Worker (safety net only)	Verified: TP-F02 (W10; `Confirmed`/`Processing` since the review, Section 10.11)
Overload	Nginx rate limit, application concurrency limit, bounded queue; reject before creating an order, bounded Outbox backlog and bounded unfinished orders; drain and recovery after overload	Verified: TP-L01, TP-L02 (W12); TP-L04 local (after the review) and on EC2 (earlier code, W12)

Known gaps in the supported model. None remain open in the delivery path. An `OrderProcessed` that arrives while the order is still `PendingStock` used to be

dead-lettered. Since Week 9 it is requeued onto delayed retry queues and applied once the order is `Confirmed` (TP-M02). The database-error defect, where an error was treated as a duplicate and the result lost, was fixed in Inventory Service in Week 7 (TP-I03) and in Order Service in Week 8 (TP-W03).

Not supported (out of scope). Redis, RabbitMQ or PostgreSQL cluster failover or replication; loss of Redis data without re-running the warm-up; payment failure and compensation (the Process Worker is success-only); multi-region or multi-node failures; authentication-related failures.

10 Implementation

10.1 Baseline Configurations — C0 and C1 (Week 3)

Before building the real asynchronous design (Weeks 5–12), two controlled baseline configurations were built inside Order Service, with no separate Inventory Service involved. Both use one raw-SQL table, `order_service.inventory`. It is deliberately *not* EF Core-migrated (see `docs/erd.md`), because C0 and C1 are baseline scaffolding, not the final design. Every result in this section was produced on 2026-10-01 from a fresh clone of the repository and a brand-new PostgreSQL database built by `scripts/verify-environment.ps1`, with Order Service connecting under its own account (`order_service_user`). The method and pass rules are in `docs/baseline-c0-c1.md`; the raw evidence is in `tests/baseline/results/` and `tests/jmeter/results/`.

10.1.1 Reset and seed

`scripts/reset-and-seed.ps1` applies the baseline schema, empties both baseline tables and inserts the starting inventory: `c0-demo-product` = 1, `c1-demo-product` = 1 and `flash-product-1` = 1000, with no other rows and no order rows. `scripts/verify-reset-seed.ps1` proves the script is repeatable. It runs it, then deliberately breaks every table (wrong stock, a deleted seed row, an extra product and 25 leftover order rows), then runs it twice more. All three snapshots matched the expected state exactly and were identical to each other (`20261001T090034Z-reset-seed.log`).

10.1.2 C0 – the naive baseline (a controlled failure demonstration)

C0 (`src/FlashSale.OrderService/Experiments/C0Naive/C0NaiveDemo.cs`) implements the textbook race condition. It reads the current stock, checks it in

application code, then issues a separate, unconditional UPDATE: two round trips with no locking between them.

```
SELECT stock FROM order_service.inventory WHERE product_id = @p;
-- (application code checks: stock > 0 ?)
UPDATE order_service.inventory SET stock = stock - 1 WHERE product_id = @p;
```

C0 has no HTTP route. It runs only via `dotnet run -- --run-c0-demo`, which bypasses the ABP host, so no stray request can reach it later in the project. Five concurrent attempts were fired at a product seeded with `stock = 1`. All five wait on a shared start gate so they reach the read at the same instant, and a 50 ms pause between read and write widens the race window so it reproduces on every run. The captured result (`20261001T085538Z-c0-race.log`):

```
[C0] Seeded 'c0-demo-product' with stock = 1. Firing 5 concurrent naive
↪  requests...
[C0] Attempt 4: saw stock=1, Confirmed.
[C0] Attempt 1: saw stock=1, Confirmed.
[C0] Attempt 3: saw stock=1, Confirmed.
[C0] Attempt 2: saw stock=1, Confirmed.
[C0] Attempt 5: saw stock=1, Confirmed.
[C0] Result: 5 of 5 concurrent requests got 'Confirmed' for a product seeded
↪  with stock = 1.
[C0] OVER-SOLD: more than one request reserved the same single unit of stock.
↪  This is the bug C1 (Step 3.3) fixes.
[C0] Final stock in database: -4 (started at 1).
```

All five requests believed they had reserved the single unit, and the database was left at -4. This is the race condition C1 exists to eliminate. `order_service.inventory` deliberately has no `CHECK (stock >= 0)` constraint, so that the bug shows up as overselling instead of being turned into an unhandled exception.

10.1.3 C1 – the PostgreSQL atomic update

C1 (`src/FlashSale.OrderService/Controllers/BaselineOrdersController.cs`, `POST /api/c1/orders`) replaces the two round trips with one conditional statement. Its `WHERE` clause is evaluated by PostgreSQL against the row's current value while holding the row lock, not against a value the application read earlier:

```
UPDATE order_service.inventory
SET stock = stock - 1
WHERE product_id = @productId AND stock >= 1
RETURNING stock;
```

The update and the outcome row in `baseline_orders` commit in one transaction, so the log cannot disagree with the stock. A separate `SELECT ... FOR UPDATE` followed by an `UPDATE` was avoided, because it re-creates C0's two-statement window unless written very carefully. C1 lives on its own route; `/api/orders` belongs to the asynchronous design, so both configurations stay independently reachable for the Week 13–14 comparison.

Concurrent demand below, equal to and above stock. `scripts/run-baseline.ps1` seeds `c1-demo-product` with 100 units, starts every request before reading any response, and repeats each scenario three times:

Scenario	Requests	Confirmed	Rejected	Final stock	HTTP 200	Verdict
Below stock	50	50	0	50	50/50	PASS (3/3 run
Equal to stock	100	100	0	0	100/100	PASS (3/3 run
Above stock	150	100	50	0	150/150	PASS (3/3 run

In all nine runs, Confirmed equalled $\min(\text{demand}, 100)$, stock never went negative, final stock equalled 100 minus Confirmed, and `baseline_orders` held exactly one row per request (`20261001T085538Z-c1-*-run*.json`).

Connection-pool finding. The first concurrent prototype never oversold, but 3–31 of every 150–300 requests failed with HTTP 500 (`53300: sorry, too many clients already`). Npgsql's default pool of 100 connections per service is larger than what PostgreSQL's default `max_connections = 100` leaves for ordinary roles (97), and both services share that server. The pools are now capped at 60 (Order Service) and 30 (Inventory Service). These sizes are part of the experiment configuration and are recorded with every run.

10.1.4 Correctness criteria

The invariants in `docs/inventory-invariants.md` are applied as follows. Invariant 1 (`available_inventory >= 0`) and Invariant 2 (`successful_reservations <= initial_inventory`) are checked after every run from the database itself: final stock, and Confirmed rows in `baseline_orders`. C0 violates both under concurrency (final stock -4, 5 successes for 1 unit). C1 satisfied both in all nine runs. The captures add two run-level checks: every request receives a decision (HTTP 200 with `Confirmed` or `Rejected`, and no errors), and stock is conserved (final stock = initial stock − Confirmed). Invariants 3 and 4 (one reservation per order; no duplicate effect from a redelivered message) need an order identity and

message delivery. C0 and C1 have neither, so those invariants are checked on the asynchronous design from Week 7 onward.

10.1.5 JMeter smoke test

The checked-in plan `tests/jmeter/smoke-test.jmx` runs 10 threads with one `POST /api/c1/orders` each for `flash-product-1`. Each sample carries two assertions: HTTP status 200, and a body that is a `Confirmed` decision for that product with a `remainingStock` value. The previously committed result file had been recorded against `/api/orders`, the asynchronous endpoint, so it did not match the plan. It was removed and replaced. `scripts/run-jmeter-smoke.ps1` resets and seeds the database, runs the plan with JMeter 5.6.3, saves a timestamped JTL and cross-checks it against the database.

Result (`tests/jmeter/results/20261001T090632Z-c1-smoke.jtl`): 10 of 10 samples hit `/api/c1/orders`, all returned 200 and passed both assertions, the database recorded exactly 10 Confirmed C1 decisions, and stock fell from 1000 to 990. Elapsed time was 28 ms minimum, 36.5 ms median and 162 ms maximum. These figures come from one local smoke run and are a functional check, not a performance result. A control run against a product with no stock still returned 200 but failed the content assertion on all 10 samples, which shows the assertions can fail.

10.2 Order Service

Order Service accepts orders, stores them with their `OrderPlaced` Outbox row, answers status polling, and applies the stock and processing results that come back as events. The behaviour below is specified in `docs/order-api.md`. Every claim in this section is backed by the saved Week 5 evidence (`tests/evidence/20261001T123816Z-*`), produced from a fresh clone on a brand-new database with Order Service running under its own database account.

10.2.1 API design

Request	Status	Meaning
`POST /api/orders`	201	New order in `PendingStock`; body is the order with its id and `requestAcceptedAt`
	200	Same `Idempotency-Key` and same body: the original order, unchanged
	400	Missing or blank key, quantity $\neq 1$, blank product, or malformed JSON; nothing is stored
	409	Key already used with a different body; the original order is unchanged
`GET /api/orders/{id}`	200	Current state and the stage timestamps
	404	Unknown order id

The request carries `productId` and `quantity`. Quantity must equal 1, because flash-sale orders are single-unit and the reservation script decrements exactly one unit. A product id with no stock is *not* a validation error: the order is accepted, and Inventory Service rejects it asynchronously. The order row and its `OrderPlaced` Outbox row are written in one `SaveChangesAsync` call (Section 10.2.5). All timestamps are UTC with a `Z` suffix and are stored at microsecond precision, so a value read back from PostgreSQL equals the value returned at creation. Validation runs before any write, so a rejected request leaves no order and no Outbox row.

10.2.2 Idempotency strategy

The client sends an `Idempotency-Key` header. It is the *transport* key for client retries, and is distinct from the broker's `MessageId`, which the Inbox uses to absorb redelivered messages (Section 9.7). The key is enforced at two levels:

- **Lookup before insert.** A known key with the same body returns the original order (200); with a different body it returns 409.
- **Unique index as the final arbiter.** Concurrent requests with the same key can all miss the lookup. Only one insert can succeed against `IX_orders_IdempotencyKey`. A request that loses this race catches exactly that unique violation (PostgreSQL `23505` on that constraint, nothing broader), re-reads the winning order and answers as a replay or a conflict.

What the Week 5 check found. Before the fix, quantities 0, 2 and -1 and an empty product id were accepted as orders. When 20 requests with the same

key were sent at once, 6 to 9 of them returned HTTP 500: the unique index prevented duplicate rows, but the losing requests crashed. A replayed order also came back with different timestamps (no `Z`, different precision). On a fresh clone, an unknown order id returned 500 instead of 404, because ABP's themed error page needs website files that are not in the repository; the error page now applies only outside `/api`. `scripts/verify-order-api.ps1` fails 10 checks on the old code.

Result after the fix (`20261001T123816Z-order-api.log`, 30 of 30 checks):

- every validation case returns 400 with no row stored;
- a replay returns 200 with a body identical to the first response, and a changed body returns 409 with the original order unchanged;
- in three rounds of 20 concurrent same-key requests, each round gave exactly one 201 and nineteen 200s naming the same order, with one order row and one Outbox row;
- 200 concurrent requests over 50 keys (4 per key) produced exactly 50 orders and 50 Outbox rows, one 201 per key, and no key with more than one order in the table.

10.2.3 State management

`Entities/Order.cs` implements the state machine as guarded transitions (`Order.TransitionTo`). The current code has three legal transitions: `PendingStock` → `Confirmed`, `PendingStock` → `Rejected` and `Confirmed` → `Completed`. Every other pair throws `InvalidOrderStateTransitionException` and changes nothing. The adopted six-state model adds `Processing` in Week 11 (Section 9.5).

Result events are applied by `StockResultProcessor`:

- an event for the state the order is already in, with any `MessageId`, is acknowledged without a transition and recorded in the Inbox;
- an event for a state the order has already passed on `PendingStock` → `Confirmed` → `Completed` (for example `StockReserved` after `Completed`) is likewise acknowledged. Before Week 5 this case threw and would have been dead-lettered; it was found by the Week 5 code check and fixed;
- a conflicting event (for example `StockRejected` after `Confirmed`) throws, nothing is recorded, and the consumer sends it to the dead-letter queue;

- an event for an unknown order is acknowledged and not recorded.

An `OrderProcessed` that arrives while the order is `PendingStock` used to be dead-lettered instead of retried. That was fixed in Week 9 (TP-M02): it is requeued until the order is `Confirmed`.

10.2.4 Test cases

Case	What it shows	Status
TP-A01	Intake returns 201 `PendingStock`; polling returns the order; unknown id is 404	PASS
TP-A02	10 invalid requests (key, quantity, product, JSON) return 400 and store nothing	PASS
TP-A03	Replay returns the identical original order	PASS
TP-A04	Changed body returns 409; original unchanged	PASS
TP-A05	Concurrent same-key retries: one 201, the rest 200, no 5xx	PASS
TP-A06	Full 4×4 transition matrix; duplicate, stale, conflicting and unknown-order events	PASS
TP-A07	50 keys $\times$ 4 concurrent requests: exactly 50 orders and Outbox rows	PASS

TP-A01–A05 and TP-A07 are run by `scripts/verify-order-api.ps1` against the running service. TP-A06 consists of the unit tests in `OrderStateBehaviourTests`. The whole unit suite passes, 43 of 43, in the saved run `tests/evidence/20261001T123816Z-unit-tests.trx`. `docs/test-plan.md` lists the inputs, expected outcomes and commands.

```
Passed!  - Failed: 0, Passed: 43, Skipped: 0, Total: 43
```

10.2.5 Event Publication Reliability (Week 6) – the dual-write problem and the Transactional Outbox

The problem. Once order creation needs to notify another service (Inventory Service, Week 7) that stock should be reserved, the order write and the "notify" action become two separate operations against two separate systems (Postgres and RabbitMQ) that cannot be wrapped in one atomic transaction the way two SQL statements against the same database can. Naively sequencing them — write

the order, then publish an event — exposes two distinct failure orderings, and a system that only handles one of them is still broken:

1. **The database commits but the publish fails.** The order now exists in `PendingStock` with no corresponding event ever sent. Nothing will ever move it out of `PendingStock` – it is silently stuck forever, and neither the client nor any operator has any signal that anything is wrong.
2. **The publish succeeds but the database transaction rolls back** (or the process crashes between the two steps). Inventory Service now receives an `OrderPlaced` event for an order that, as far as Order Service's own database is concerned, never happened. It may reserve real stock against a phantom order that no client will ever be able to query or complete.

Both failures share a root cause: two systems that must agree with each other are being updated as if they were one, with no mechanism enforcing that they actually end up agreeing.

How the Outbox pattern eliminates both. Rather than publishing directly, the order-creation handler writes two rows to *one* database – the `orders` row and a row in `order_service.outbox_events` recording that an `OrderPlaced` event needs to be published – inside the exact same `SaveChangesAsync` call (`Controllers/OrdersController.cs`). Because both rows are ordinary rows in the same relational database, Postgres's own transaction guarantee applies to both simultaneously: either both commit, or neither does. This directly closes both failure orderings above — there is no longer a window where the order exists without a corresponding pending event, or vice versa, because "the order exists" and "an event needs publishing" have been reduced to a single fact instead of two facts that could disagree.

This claim is verified directly, not just argued from the pattern's description, in `tests/FlashSale.OrderService.Tests/OutboxAtomicityTests.cs`: a real SQLite in-memory database (not a mock, and not EF Core's InMemory provider, which does not enforce relational constraints and so cannot demonstrate a rollback at all) is made to reject an outbox insert via a genuine primary-key collision, and the accompanying order insert is confirmed absent afterward — the whole transaction rolled back, not just the failing half.

```
Failed_outbox_insert_rolls_back_the_order_insert_too: PASS
Order_and_outbox_row_both_commit_in_one_SaveChanges_call: PASS
```

Both pass in the saved run `tests/evidence/20261001T123816Z-unit-tests.trx` (TP-E03).

The same guarantee on the real database. The unit test uses SQLite. `scripts/verify-outbox.ps1` repeats the check on PostgreSQL with Order Service running under its own account (`tests/evidence/20261001T163530Z-outbox.log`, 11 of 11 checks). A normal request stores exactly one order and one unpublished `OrderPlaced` row. With the Outbox `INSERT` privilege revoked for one request, the request failed and *no* order row existed afterwards. Once the privilege was restored, the same key created the order normally.

Outbox workflow. Writing the Outbox row only records that an event must be published. A background worker (`OutboxPublisherWorker`) polls every 2 seconds for unpublished rows, oldest first, 50 at a time, and publishes each stored payload. The event's `MessageId` is generated once, when the Outbox row is written, and stored in the payload, so every publish attempt for that row sends the same value. A failed attempt is retried after 1 s, 2 s and 4 s. After that the row stays unpublished and is tried again on the next poll, indefinitely, until it succeeds. The worker never crashes the host: a failed tick is logged and the loop continues. In the saved run, the broker was unreachable when the order was created. The publisher logged failed attempts and left the payload unchanged. Once a broker came up, exactly one message arrived, carrying the `MessageId` stored at commit time.

Routed confirmation. A row is marked published only when the broker has confirmed the message *and* delivered it to every queue that must receive it (`docs/outbox.md`). The Week 6 code check found that this was not true. All three publishers (both Outbox publishers and the Process Worker's) used `mandatory: false`, so a message with no matching binding was silently dropped, yet confirmed and marked published. The prototype reproduced it: an `OrderPlaced` published before any queue was bound was marked published and reached nobody. A second gap is subtler. `StockReserved` must reach two queues, Order Service's and the Process Worker's. If only one was bound, the broker accepted the message while the Process Worker never saw it.

The fix is the same in all three publishers. The queues each event must reach are configured per event (`RequiredSubscriberQueues: OrderPlaced` → `OrderPlaced`; `StockReserved` → `StockReserved` and `ProcessWorker.StockReserved`; `StockRejected` → `StockRejected`; `OrderProcessed` → `OrderProcessed`). Before each publish, every required queue is checked with a passive declare, which fails if it does not exist, and is then bound to the exchange; binding is idempotent. The message is published with `mandatory: true` on a channel with publisher confirmations and tracking. `BasicPublishAsync` then throws on a nack or a return. Any exception leaves the row unpublished, and an event type with no configured

queues is refused. The consumers still own their queues; the publisher never creates one.

Failure scenarios. `scripts/verify-publication.ps1` runs each scenario against a throwaway broker (`tests/evidence/20261001T163055Z-publication.log`, 18 of 18 checks). On the old publishers, 8 of its checks fail: the affected messages were marked published but were never delivered.

Scenario	Observed with the fix	Old code
Valid route	Published; one message in the queue	Same
Required queue missing	Row stays unpublished, retries logged; once the queue exists (even unbound) it is delivered once and the binding is restored	Marked published, message lost
Binding removed	Binding restored; delivered once	Marked published, message lost
Broker stopped, then restarted	Orders still accepted; row unpublished while down; delivered once after restart	Marked published, message lost
Full queue (`reject-publish`)	Second message nacked and left unpublished; delivered once after space freed	Same (a nack was already detected)
Unroutable message	`PublishException` with `IsReturn = true` (broker probe)	No exception
`StockReserved`, Process Worker queue missing	Not published, nothing sent to the other queue; once both exist, one copy in each	Marked published, Worker copy lost
`OrderProcessed` queue missing	Worker retries, then dead-letters its `StockReserved`; once the queue exists, one `OrderProcessed` with `MessageId = OrderId`	Not run on the old code

In every recoverable case the event arrived exactly once after the fault was removed, without manual intervention. A Process Worker event that cannot be published is not lost: it lands in the worker's dead-letter queue, and recovering it from there is Week 10's subject. ABP's own RabbitMQ event bus was not used

for publishing because its options expose no documented way to enable publisher confirms; the raw `RabbitMQ.Client` 7.1.2 package is used instead.

10.3 Inventory Service

10.3.1 Scaffolding

Inventory Service was scaffolded with a minimal ABP module set (MVC, Autofac, EntityFrameworkCore.PostgreSql, Swashbuckle, Serilog). It has none of the identity, account, permission, tenant, setting or OpenIddict modules that Order Service carries from the default template: the scope lock excludes user management, authentication and multi-tenancy, so Inventory Service never adds them. It owns only the `inventory_service` schema (Section 9.7) and the Redis inventory keys. The behaviour below is specified in `docs/inventory.md`. Every result in this section comes from saved runs on a fresh clone with throwaway Redis, RabbitMQ and PostgreSQL instances (`tests/evidence/20261002T051418Z-inventory.log` and `20261002T052244Z-cross-store.log`).

10.3.2 Redis key and data design

Key	Type	Meaning
`inventory:{productId}`	integer	Units still available
`processed:{productId}`	set	Order ids that hold a reservation for this product
`sale:open:{productId}`	string	Present only after a confirmed warm-up: the sale is open

Stock lives only in Redis for the asynchronous design. The per-product reservation set lets the duplicate check happen inside Redis, in the same atomic step as the stock change, without consulting PostgreSQL.

10.3.3 Lua reservation logic

`scripts/redis/reserve.lua` runs as one atomic `EVAL`: Redis executes a script without interleaving any other command, so the check-then-act sequence below cannot race. The C# side (`InventoryReservationService`) only loads the file and passes keys and arguments; it does not reimplement any of the logic.

```
-- KEYS: inventory:{id}, processed:{id}, sale:open:{id}   ARGV: order_id
```

```
if redis.call('SISMEMBER', KEYS[2], ARGV[1]) == 1 then return 'DUPLICATE' end
if redis.call('EXISTS', KEYS[3]) == 0 then return 'NOT_OPEN' end
local stock = tonumber(redis.call('GET', KEYS[1]))
if stock == nil or stock <= 0 then return 'REJECTED' end
redis.call('DECR', KEYS[1])
redis.call('SADD', KEYS[2], ARGV[1])
return 'RESERVED'
```

The duplicate check comes first, so a redelivered order that already holds a unit is answered `DUPLICATE` even if the sale has since closed. Inventory Service maps `RESERVED` and `DUPLICATE` to `StockReserved`, and `REJECTED` and `NOT_OPEN` to `StockRejected`.

10.3.4 Inventory warm-up and readiness

The teacher brief requires that a product opens for sale only after warm-up is complete *and confirmed*. The Week 7 code check found no such gate. An unwarmed product was answered `REJECTED` exactly like a sold-out one, and nothing recorded whether warm-up had happened.

`scripts/warm-up.ps1` now handles each product in one atomic Lua step. It sets the stock, clears the reservation set and sets `sale:open:{productId}`. It then confirms the product by reading all three back, prints `CONFIRMED` per product, and exits non-zero if any product does not confirm. Until the marker exists, `reserve.lua` answers `NOT_OPEN` and touches no stock. Re-running warm-up on a product whose sale is open is refused, because it would restore stock and forget reservations mid-sale. `-Force` exists for experiment resets only.

In the saved run, a reservation attempted before warm-up answered `NOT_OPEN` with the stock untouched. Warm-up confirmed all six test products. A re-run on an open sale was refused (stock stayed at its post-sale value) and `-Force` reset it. End to end through the running Inventory Service, an `OrderPlaced` for a never-warmed product produced `StockRejected` with no Redis key created, and one for a warmed product produced `StockReserved`.

10.3.5 Inventory invariants under concurrency

For every product, after any amount of concurrent traffic (`docs/inventory.md`): (1) `inventory` ≥ 0; (2) reservations $\leq$ initial stock; (3) `inventory` + reservations $=$ initial stock (every unit is either available or held by exactly one order); (4) no order id is reserved twice, and a repeated order id is answered `DUPLICATE` only if that order already holds a unit. `tests/InventoryProbe` drives the real `reserve.lua` with all requests in flight at once and checks these per product:

Workload	Requests	Reserved	Duplicate	Rejected
Hot product: 100 units; 300 orders + 100 repeats	400	100	52	248
4 products: 40/30/20/10 units; demand 50/25/15/10 %; 250 orders + 80 repeats	330	100	45	185

All four invariants held for every product in both workloads. The number of `RESERVED` answers equalled the reservations recorded in Redis, and equalled $\min(\text{distinct demand}, \text{stock})$ for every product. Stock ended at exactly zero wherever demand exceeded supply. The probe also fails when it should: run before the sale was open, every request answered `NOT_OPEN` and the conservation checks failed.

10.3.6 OrderPlaced handling and the Redis/PostgreSQL boundary

`OrderPlacedConsumer` consumes with manual acknowledgement. `OrderPlacedProcessor` checks the Inbox, runs `reserve.lua`, then writes the result event to its Outbox and the Inbox row in one PostgreSQL transaction. Only after that commit is the message acknowledged. A processing error is retried after 1 s, 2 s and 4 s, and then the message is dead-lettered (`OrderPlaced.dlq`), never re-queued in a loop.

PostgreSQL failure after the Redis reservation. Redis and PostgreSQL cannot share a transaction, so the design makes Redis the decision and PostgreSQL the record (Section 9.7). The Week 7 test (`scripts/verify-cross-store.ps1`) revokes the Outbox `INSERT` right after `reserve.lua` has run. On the old code the consumer caught *every* database error as "a duplicate already recorded this" and acknowledged the message. The `StockReserved` result was then lost for good while the unit stayed reserved, and 5 of the test's 11 checks failed. The processor now treats only a unique-key violation (PostgreSQL `23505`) as a duplicate; any other error propagates to retry and the dead-letter queue. With the fix (11 of 11 checks):

- **Short outage.** The privilege is restored inside the retry window. The retry reached `reserve.lua` again, got `DUPLICATE`, and wrote `StockReserved`. Stock was deducted exactly once.

- **Long outage.** The retries were exhausted and the message went to `OrderPlaced.dlq` with no Inbox row and no result. After the fault was removed, replaying the dead letter rebuilt `StockReserved`, again with exactly one deduction.

Order Service's `StockResultProcessor` had the same catch-all; it was closed in Week 8 (TP-W03).

10.4 Closing the Loop – Stock Results and Order State (Week 8)

Weeks 6–7 built one direction of the pipeline: an order becomes an `OrderPlaced` event, which Inventory Service reserves against Redis. Week 8 builds the return direction – Inventory Service telling Order Service what happened, and Order Service acting on it – which is what actually lets an order leave `PendingStock`.

10.4.1 Inventory Service's own Outbox – a harder dual-write problem

Order Service's Week 6 Outbox writes a business row and an outbox row to the *same* Postgres database in one transaction, so atomicity is free. Inventory Service cannot do that: the business fact (stock was reserved) is recorded in Redis by `reserve.lua`, and the outbox fact (tell Order Service) must be recorded in Postgres – two different data stores that cannot share a transaction. Rather than accept a gap where a process crash between the two writes silently drops the notification, Inventory Service gets its own `outbox_events` table (`inventory_service` schema) with one deliberate difference from Order Service's: the unique index is on `OrderId`, not `Id`.

That difference is what makes the Lua script's `DUPLICATE` outcome safe. `DUPLICATE` means a *previous* delivery of the same `OrderPlaced` message already reserved the stock in Redis – so if the consumer treated it as a no-op, a redelivered message would never produce a `StockReserved` row, and that order would sit in `PendingStock` forever despite the reservation having genuinely succeeded. Instead, both `RESERVED` and `DUPLICATE` run the same step in `OrderPlacedProcessor`: check whether an outbox row for this `OrderId` already exists, and only insert if it doesn't. The unique index turns a possible race between two redeliveries into a unique violation, which is caught, rather than a duplicate row – "ensure exists", not "insert unconditionally". The Inventory Service `RabbitMqOutboxPublisher` and

`OutboxPublisherWorker` use the same design as Order Service's Week 6 versions (raw `RabbitMQ.Client`, routed confirmation, 2s poll interval).

10.4.2 Order Service: consuming the result

`StockResultConsumer` consumes two queues, `StockReserved` and `StockRejected`. Each queue has its own dead-letter queue. Both queues share one handler, which calls `StockResultProcessor` with the target state. The processor works in this order:

1. Check the Inbox. If the `MessageId` has already been processed, acknowledge and stop.
2. Look up the order. If it does not exist, acknowledge and stop.
3. If the order is already in the target state, or has moved past it on the happy path, record the Inbox row and treat the message as a no-op.
4. Otherwise call `Order.TransitionTo` and add the Inbox row. Both are committed by one `SaveChangesAsync`.

The message is acknowledged only after that commit. Failures are handled in three ways:

- **Conflicting result.** For example, `StockReserved` for a `Rejected` order. This is not transient, so it is dead-lettered at once.
- **Database error.** It is retried after 1, 2 and 4 seconds, then dead-lettered. Only a unique violation counts as a concurrent duplicate (TP-W01, TP-W03).
- **`OrderProcessed`.** The Process Worker's event is handled by `OrderProcessedConsumer`. It reuses the same processor with the target state `Completed`.

10.4.3 Automated correctness validation

`scripts/validate-correctness.ps1` checks the four inventory invariants for one product after a run, with separate rules for C1 and C2 (`docs/validation.md`). It takes the configuration, product id and initial stock as parameters, so Weeks 13–14 can reuse it unmodified.

What the Week 8 code check found. The earlier validator gave false passes. It could not validate C1 at all, because it read Redis and the asynchronous `orders` table while C1 lives in `order_service.inventory` and `baseline_orders`. Two of its four checks could never fail. Invariant 3 counted duplicate order *primary keys*, which the primary key forbids. Invariant 4 counted duplicate `MessageIds` in `processed_messages`, which its unique index forbids. On a fixture where 3 units were taken for 2 reserved orders (a double deduction), the old validator printed `PASS` for all four invariants and exited 0. The validator results shown in earlier drafts of this report were produced by that version, and their invariant 3 and 4 lines are therefore not evidence.

Rules. For C1: stock ≥ 0; Confirmed decisions $\leq$ initial; units taken (initial $-$ stock) $=$ Confirmed decisions, so every deduction has exactly one logged decision. Invariant 4 is reported `NOT APPLICABLE` with its reason, because C1 has no broker messages. For C2: Redis stock ≥ 0; both Redis reservations and successful orders $\leq$ initial. The Redis reservation set and the successful orders must be exactly the same order ids (one reservation per order). Units taken must equal distinct reserved orders (no double deduction), and the Inbox `MessageId` uniqueness must exist in both services. A run with no data, or with a reserved order still `PendingStock`, is `INCONCLUSIVE`; an unreachable store or a failed query is `SETUP ERROR`. Both exit non-zero, so nothing unchecked is ever reported as a pass. Since Week 12, C3 is validated from the same evidence as C2, and a fifth check accounts for every accepted order: each order row is either terminal (`Completed`, `Rejected`) or outstanding (`PendingStock`, `Confirmed`, `Processing`), and the two add up to the orders accepted. An outstanding order makes the check `INCONCLUSIVE` and is listed with its age and re-publish count, so no accepted order can go unnoticed. The broken-fixture test now also covers check 5 under C3 (16 of 16 checks, `tests/evidence/20261020T030140Z-validator.log`).

Evidence (fresh clone; `tests/evidence/20261002T062442Z-validator.log` and `20261002T062442Z-real-runs.log`). `scripts/verify-validator.ps1` feeds the validator correct C1 and C2 fixtures, which pass, and one broken fixture per failure mode: oversold stock, a deduction without a logged decision, a reservation/order mismatch, a double deduction, an unsettled run, missing data and an unreachable database. Each broken fixture fails the targeted invariant or is reported `INCONCLUSIVE` or `SETUP ERROR` (12 of 12 checks). `scripts/verify-real-runs.ps1` then validates two real runs.

- **C1:** 30 concurrent requests against 20 units. All four invariants hold: stock 0, 20 Confirmed.

- **C2:** Order Service, Inventory Service and the Process Worker running together. 30 orders plus 10 client retries against 20 units settled as exactly 30 orders: 20 `Completed` and 10 `Rejected`. The Redis reservation set and the 20 successful orders were the same order ids, and stock was 0.

While this script was being developed, a second run reused a product id whose orders were still in the database from the first run. The validator correctly failed it, because the successful orders exceeded the stock that had been re-seeded into Redis.

10.4.4 The main workflow (Step 8.4)

`docs/workflow.md` describes the one business workflow, placing an order, from the Week 8 code. Its steps are the numbered arrows of Figure 9.7. The order is accepted when its row commits, so the client receives `201 Created` with `PendingStock`. All later steps run asynchronously, and every consumer acknowledges only after its own transaction commits. Table 10.1 lists each step with its failure branch.

What the code check found. The earlier diagrams did not match the code in five places:

- They drew Nginx in front of Order Service, but Nginx forwards nothing yet.
- They drew a plain publish and confirm, without the required-queue check and the `mandatory` flag added in Week 6.
- They left out the Inbox steps.
- They left out the Week 7 warm-up gate (`NOT_OPEN`).
- They drew the two consumers of `StockReserved` one after the other, although they run concurrently.

This section had the same problem. It described a handler that nacked with requeue and "no DLQ yet". Both points have been out of date since the Week 10 retry and DLQ code. The diagrams and the text are now rewritten from the code.

The concurrent fork. The two `StockReserved` consumers are independent. If `OrderProcessed` reaches Order Service before step 22 commits, the order is still `PendingStock`. The transition to `Completed` is not yet legal. Until Week 9 the message was then dead-lettered. Now Order Service writes nothing for it, republishes it onto a delayed retry queue (2, 4, then 8 s) and applies it once the order is `Confirmed` (TP-M02).

Steps	What happens	If it fails	Test
1–4	Validate; look up the `Idempotency-Key`; insert the order and its `OrderPlaced` Outbox row in one transaction; answer `201`.	`400`/`409`, or a replay answered `200`; a refused Outbox insert leaves no order.	A01–A07, O01
5–9	Every 2 s, publish up to 50 unpublished rows (oldest first) after checking queue `OrderPlaced`; mark published after the routed confirm.	Not marked published; retried after 1, 2, 4 s, then next poll; same `MessageId`.	O01, O02
10–15	Inventory checks its Inbox, runs `reserve.lua`, then writes the result's Outbox row and its Inbox row in one transaction, then acks.	Retried, then dead-lettered; a replay gets `DUPLICATE`, so there is no second deduction.	I01–I03
16–20	Publish `StockReserved` (queues `StockReserved` and `ProcessWorker.StockReserved`) or `StockRejected`.	As for steps 5–9.	O02
21–23	Order Service: Inbox check, `PendingStock` to `Confirmed/Rejected` and the Inbox row in one transaction.	Retried, then dead-lettered; a conflict is dead-lettered at once.	W01, W03
24–27	Process Worker: fixed 200 ms delay, publish `OrderProcessed` with `MessageId` = `OrderId`.	Retried, then dead-lettered; a redelivery republishes the same `MessageId`.	P01 (W11)
28–30	Order Service: `Confirmed` to `Completed` with the Inbox row in one transaction.	If the order is still `PendingStock`: requeued (2, 4, 8 s), then applied.	W04, M02
31–32	`GET /api/orders/{id}` returns the state.	`404` for an unknown id.	A01

Table 10.1. – Steps of the main workflow (test case ids without the `TP-` prefix).

Evidence (TP-W04; fresh clone; `tests/evidence/20261002T152659Z-workflow.log`). The verifier passes its 35 static checks. Each check is tied to the code or configuration. For example:

- Each event's drawn required queues must equal the `RequiredSubscriberQueues` setting.
- The drawn processing delay must equal the configured one.
- The diagram files in `report/figures/` must be identical to the diagrams in the document.

The previous diagrams fail 20 of these checks. Changing one setting, such as the publish flag, the worker delay or a required queue, fails exactly one check.

The live part ran Order Service, Inventory Service and the Process Worker on a throwaway RabbitMQ and Redis, with three orders:

- **A reserved order.** A client replay of the same key returned `200` with the same order. The order reached `Completed`. Its rows appeared in the drawn order: the Order Outbox row, then Inventory's Inbox and Outbox rows, then `Confirmed` with its Inbox row, then `Completed` with the Inbox row whose `MessageId` is the order id. Redis deducted one unit.
- **A sold-out order.** `REJECTED`, then `Rejected`. Nothing was deducted, and the Process Worker never saw it.
- **An order for a product that was never warmed up.** `NOT_OPEN`, then `Rejected`. No Redis key was created.

At the end, every queue and every DLQ was empty (27 of 27 live checks). Only orderings that cause and effect guarantee are compared, because a publisher marks a row published only after the confirm returns. Reconstructing the same path from the three services' logs by `CorrelationId` is the Week 11 trace (TP-P03).

Negative control. To rule out a false pass, the same live run was repeated with one deliberate break: the Process Worker was configured to require a queue that does not exist before publishing `OrderProcessed`. The verifier then failed 6 checks, all on the broken steps. The order stayed `Confirmed`, the completion step and its logs were missing, and the message was in `ProcessWorker.StockReserved.dlq` (`20261002T152659Z-workflow-negative-control.log`).

10.5 Reliability and Idempotency Layer (Week 9)

10.5.1 Message delivery semantics

RabbitMQ delivers at-least-once. A message is removed from its queue only when the consumer acknowledges it. If the consumer dies first, or its connection drops, the broker delivers the same message again. Duplicates therefore cannot be avoided at the transport. The system's goal is the next best thing: every side effect happens effectively once, however many times a message arrives.

Two kinds of duplicate have to be absorbed, by two different layers.

- **Transport duplicates.** The same broker message (same `MessageId`) arrives again, for example after a crash or a connection loss. The *Inbox* absorbs it. Each service has a `processed_messages` table with a unique index on `MessageId`, created by the `AddProcessedMessages` migration of each service.
- **Business duplicates.** The same fact arrives under a *new* `MessageId`. The reconciliation job does exactly this when it re-publishes `OrderPlaced`. The Inbox cannot see these, because the `MessageId` is new. Business rules absorb them instead:
  - Redis's `processed:{product}` set makes the Lua script answer `DUPLICATE` for an order that already holds a unit;
  - Inventory's result Outbox has one row per order id;
  - Order Service treats a result for a state the order already has, or has passed, as a no-op.

Table 10.2 lists every consumer. Only a PostgreSQL unique violation (23505) is treated as "already processed". Any other database error is retried and then dead-lettered, so a failed write is never acknowledged as done (Week 8, TP-W03).

The Process Worker has no Inbox because it writes nothing locally. A redelivered `StockReserved` makes it publish the same `OrderProcessed` again, with the same `MessageId`. Order Service's Inbox then absorbs that copy.

10.5.2 Idempotent consumer

The correctness logic lives in two small classes that a test can drive directly. `OrderPlacedProcessor` serves Inventory Service, and `StockResultProcessor` serves both Order Service consumers. The consumer classes around them only deserialize, retry, and acknowledge or reject. Each processor works in this order:

Consumer	Inbox	One local transaction	Acknowledged
Inventory: `OrderPlacedConsumer`	Inventory's	result Outbox row (one per order) and Inbox row	after the commit
Order: `StockResult Consumer`	Order Service's	state change and Inbox row	after the commit
Order: `OrderProcessed Consumer`	Order Service's	`Confirmed`→`Completed` and Inbox row; an early completion writes nothing and is requeued	after the commit, or after the confirmed requeue
Process Worker: `StockReserved Consumer`	no Inbox (no database)	none: it only publishes `OrderProcessed` with `MessageId` = order id	after the confirmed publish

Table 10.2. – Every consumer, its Inbox, its local transaction and when it acknowledges (`docs/delivery-semantics.md`).

1. **Check the Inbox.** If the `MessageId` is recorded, report "already processed" and touch nothing. In Inventory Service this check runs before the Lua script, so an exact redelivery never reaches Redis.

2. **Decide.**
   - Inventory runs the Lua reservation.
   - Order Service compares the order's state with the result. A result for the same or an earlier state on the happy path (for example a late `StockReserved` after `Completed`) is a no-op. A completion that arrives while the order is still `PendingStock` returns `NotYetApplicable`: it writes nothing and the consumer requeues it. Any other mismatch is a genuine conflict, which throws and is dead-lettered.

3. **Commit once.** The business change (Inventory's result Outbox row, or Order Service's new state) and the Inbox row are added to one `DbContext`. One `SaveChangesAsync` commits both, so either both happen or neither does.

4. **Absorb the race.** Two deliveries can pass step 1 together. The unique index then rejects the second insert, and that unique violation, and only that one, is reported as "already processed".

`Order.TransitionTo` has no self-loop for `Confirmed` or `Rejected`. That is why the processor detects "already in this state" itself. Otherwise a legitimate redelivery would throw the same exception as a genuine conflict, and the two would be indistinguishable.

The local transaction was tested directly (TP-M01). With `INSERT` on the Inbox revoked, the business write rolled back with it in both services, the message was retried and then dead-lettered, and a replay applied it once.

10.5.3 Acknowledgement strategy, proven with a real crash (Step 9.3)

Every consumer follows the same rule (`docs/delivery-semantics.md`):

1. Check the Inbox.
2. Do the business step and insert the Inbox row in one local transaction.
3. Acknowledge only after that transaction commits (`autoAck: false`).

The dangerous moment is between the commit and the ack. If the process dies there, the work is done but the broker still holds the message, so it delivers it again.

What the code check found. An earlier draft of this section described a manual crash test. Its output was never saved, and it killed Inventory Service *before* it processed the message. That shows an unprocessed message is redelivered. It does not test the case this step is about: a crash *after* the commit.

Fault injection. Each of the four consumers now calls a fault-injection hook, `CrashBeforeAckIfTargeted`, between its commit and its ack:

- Inventory's `OrderPlaced` consumer;
- Order Service's stock-result consumer;
- Order Service's `OrderProcessed` consumer;
- the Process Worker.

The call does nothing unless two test settings name the consumer and one correlation id. Neither setting is present in any `appsettings.json`. When both match, the process kills itself (`Process.Kill`), with no graceful shutdown that could still acknowledge the message. Targeting one correlation id leaves all other traffic untouched.

Process Worker. The Process Worker has no database and therefore no Inbox. Its only effect is publishing `OrderProcessed` with `MessageId` equal to the order id. A redelivered `StockReserved` makes it publish the same message again, and Order Service's Inbox absorbs the copy.

Evidence (TP-M01; fresh clone and fresh database; saved as `20261002T183238Z-delivery.log` in `tests/evidence/`). `scripts/verify-delivery.ps1` passes 34 of 34 checks on the three services, a throwaway RabbitMQ and Redis, and PostgreSQL.

- **Inbox uniqueness.** Both `processed_messages` tables have a unique index on `MessageId`. A second row with the same `MessageId` is refused as a unique violation for each service's own database account.
- **Local transaction.** With `INSERT` on the Inbox revoked, the business write rolled back with it in both services: Inventory wrote no result Outbox row, and the order stayed `PendingStock`. The message was retried, then dead-lettered. After the privilege was restored, replaying the dead letter applied the message once, and Redis deducted one unit only.
- **Crash after commit, before ack, for each consumer.** The process was gone and its work was committed: the Inbox row, plus the Outbox row and Redis deduction (Inventory) or the new state (Order Service). The unacknowledged message was back in its queue. After a restart, the same `MessageId` was redelivered and acknowledged as already processed. For the Worker, `OrderProcessed` was published again and absorbed by Order Service. Each order still had one Inbox row per message, and its decision and completion timestamps were unchanged.
- **End state.** Six orders took six of ten units, Redis recorded exactly those six orders, and every queue and DLQ was empty.

Negative controls. To rule out a false pass, the same run was repeated twice, each time with one deliberate break in Inventory Service.

- **The consumer acknowledged before processing.** The crashed message was not redelivered. The run failed 24 of its 34 checks: acknowledged messages were lost, and the orders behind them never settled.
- **The Inbox check was skipped.** The redelivery was no longer recognised as already processed (exactly one check failed). The Lua script's `DUPLICATE` answer still prevented a second deduction, which shows that Inventory is protected by more than one layer.

10.5.4 Duplicate handling

Duplicates are handled at three levels, and each level has its own test.

- **Unit level.** `DuplicateMessageIdempotencyTests` drives `StockResultProcessor` on a real SQLite engine, because the unique index is the thing under test and EF Core's in-memory provider does not enforce it. The tests cover:
  - a redelivered message, which is a no-op with one Inbox row;
  - two different messages for an order already in its target state, which are both absorbed;
  - a genuine conflict, which throws and records nothing;
  - an early completion, which is not applied and not recorded, and is then applied exactly once after `StockReserved`.

  The suite has 45 tests, and all pass in `tests/evidence/20261003T181814Z-unit-tests.log`.
- **Crash and redelivery.** A consumer killed after its commit and before its ack replays safely (Section above, TP-M01).
- **Concurrent, business-level and out-of-order duplicates.** These are covered on the running system, as described below (TP-M02).

An earlier draft showed the old validator reporting all four invariants as `PASS` on the state left by a crash test. That validator's invariant 3 and 4 checks could not fail (Section 10.4.3), so the output was withdrawn. The crash case is now covered by TP-M01 above.

10.5.5 Concurrent duplicates and out-of-order completion (TP-M02)

Three ordering risks remain once each consumer has an Inbox:

- many copies of one message arriving at the same moment;
- the same business fact arriving again under a new `MessageId` (the reconciliation job does exactly this);
- a completion arriving before the result it depends on.

What the code check found. The third case was the open gap of Section 9.7. The Process Worker and Order Service both consume `StockReserved` in parallel. So `OrderProcessed` could reach Order Service while the order was still `PendingStock`. The transition to `Completed` was then illegal, so the message was dead-lettered and the completion was lost. `scripts/verify-design-decisions.ps1` still asserted

this defective behaviour, and `docs/order-state-machine.md` described a retry that did not exist in the code.

Fix. `StockResultProcessor` has a new outcome, `NotYetApplicable`, for exactly this case. It writes nothing, not even an Inbox row, so the message can still be applied later. `OrderProcessedConsumer` then:

1. republishes the message onto a delayed retry queue (`OrderProcessed.retry.1–3`, with TTLs of 2, 4 and 8 s, each bound to the dead-letter exchange);
2. uses a channel with publisher confirms for that publish;
3. acknowledges the original only after the confirm.

Each retry queue dead-letters back onto `OrderProcessed`, and the completion is applied once the order is `Confirmed`. If it is still early after three requeues, it goes to the DLQ instead of waiting forever. The 1, 2, 4 s in-process retry for database errors is unchanged and kept separate.

Evidence (fresh clone of `b0b9d51` and fresh database; saved as `20261003T181814Z-delivery-ordering.log` in `tests/evidence/`). The verifier, `verify-delivery-ordering.ps1` in `scripts/`, passes 44 of 44 checks:

- **Concurrent duplicates.** Eight simultaneous deliveries of one `StockReserved` confirm the order once, with one Inbox row and none dead-lettered. Eight simultaneous deliveries of the same fact under eight `MessageIds` also confirm it once: all eight are recorded in the Inbox, none is dead-lettered, and the decision time is never re-stamped.
- **A new `MessageId` for the same fact.** A re-published `OrderPlaced` makes Redis answer `DUPLICATE`. No second unit is taken, and there is no second result Outbox row. A duplicate `StockReserved` is a no-op that is not dead-lettered.
- **Early completion.** `OrderProcessed` is requeued onto a bound retry queue, not dead-lettered, and writes no Inbox row while it is unapplied. It is applied exactly once after `StockReserved` lands. An orphan completion with no `StockReserved` is requeued three times, then dead-lettered.
- **Late result.** A `StockReserved` after `Completed` is a no-op, and both timestamps are unchanged.

Every order took one unit at most, and every run queue, retry queue and result DLQ ended empty.

In the same fresh clone, the earlier checks still pass: TP-M01 (34/34), TP-W04 (62/62), TP-W01 (10/10) and the unit tests (45/45, including two new tests for the early case).

Negative control. The fix was reverted in a scratch copy and the run repeated on a fresh database. 16 checks failed: the retry queues did not exist, the early completion was dead-lettered, and that order never reached `Completed`.

How it was built. DeepSeek Harness wrote the code, the documents and the verifier. Claude reviewed and re-ran every result, over three review rounds. The reviews found and returned:

- an unbound retry queue that silently lost the requeued message;
- several verifier defects;
- two hand-off claims that were false.

10.6 Failure Handling and Recovery (Week 10)

Weeks 6–9 made every individual message idempotent. Week 10 handles the failures *around* that idempotency: a handler that throws, a message nobody can ever process, and an order that slips through every other net. The mechanisms are bounded retry, a dead-letter queue per service, and a reconciliation safety net. The chapter ends with the failure model: which failures are verified, which are only designed, and which are not supported. Following the teacher brief, payment failure, compensation, Redis Cluster, RabbitMQ Cluster, PostgreSQL replication and multi-region deployment are out of scope.

10.6.1 Classify, then retry a bounded number of times

A failure is classified before it is retried (`docs/failure-handling.md`). There are three classes:

- **Transient**, such as a brief database outage or a missing queue: retried.
- **Permanent (poison)**: never retried.
- **Business rejection**, a genuine state conflict: a decision, not something to retry.

Retry is implemented directly in each of the four consumers. These are Inventory's `OrderPlacedConsumer`, Order Service's `StockResultConsumer` and `OrderProcessedConsumer`, and the Process Worker's `StockReservedConsumer`. ABP's RabbitMQ event bus is not used (Week 6).

All four use the same in-process ladder as `OutboxPublisherWorker`: four attempts, with 1 s, 2 s and 4 s waits. After the fourth attempt the consumer logs an error and nacks the message without requeue, so the message is dead-lettered with its body intact rather than looping forever. Retrying the whole processing call is safe because every processor is idempotent (Week 9): a retry that follows an attempt which had in fact succeeded sees the Inbox row and returns cleanly. Only a PostgreSQL unique violation (23505) counts as "a concurrent duplicate already recorded this". Every other database error is retried and then dead-lettered.

The two non-transient classes skip the ladder entirely:

- **Business rejection.** `InvalidOrderStateTransitionException` is deterministic, so retrying it would reproduce the same conflict four times. It is nacked to the DLQ at once.
- **Permanent (poison).** Each consumer validates the message's identity fields straight after deserialization, before any Redis, SQL or Inbox work. An empty `MessageId`, `OrderId` or `ProductId` is permanent (`OrderProcessed` carries no `ProductId`). So is an `OrderPlaced` whose `Quantity` is not 1, because the reservation scope is exactly one unit. Such a message is nacked without requeue after one attempt, and the consumer logs `Invalid <Event> message: <Field> is empty; routing to DLQ`.

The validation fixed a real bug, not just a gap. Before it, an `OrderPlaced` with an empty `MessageId` was processed as if it were real. The Inbox then recorded `Guid.Empty`, so a *second, different* invalid message was wrongly skipped as "already processed".

10.6.2 A dead-letter queue per service, and replay

Every main queue declares `x-dead-letter-exchange=flashsale.dlx`. That exchange routes each dead letter into exactly one service's DLQ:

- `OrderPlaced.dlq` for Inventory;
- `StockReserved.dlq`, `StockRejected.dlq` and `OrderProcessed.dlq` for Order Service;
- `ProcessWorker.StockReserved.dlq` for the Worker.

The Worker's queue also rewrites the dead-letter routing key. `flashsale.dlx` is a direct exchange shared by all three services. Without the rewritten key, a Worker dead letter (original key `StockReserved`) would be copied into Order Service's `StockReserved.dlq` as well. The rewrite keeps each DLQ limited to its own service's failures.

A DLQ repairs nothing by itself, so it has a documented owner and replay procedure:

1. Inspect the dead letter. Its `x-death` header names the source queue and the reason.
2. Fix the root cause.
3. Republish the message unchanged, so it keeps the same `MessageId`.

The Inbox, and `reserve.lua`'s `DUPLICATE` answer, make the replay apply exactly once.

Verification (TP-F01). `scripts/verify-failures.ps1` runs all three services on a throwaway broker and Redis. For each of the four consumers it checks four things:

- **Transient fault.** A revoked Inbox `INSERT`, or a deleted queue for the Worker, is restored inside the window. The message is applied once and nothing is dead-lettered.
- **Exhausted retries.** A fault that outlasts the ladder produces exactly four attempts, about 1 s, 2 s and 4 s apart according to the log timestamps. The message is then in that service's DLQ, with `x-death` reason `rejected`. The main queue is empty and every other DLQ is unchanged.
- **Poison.** Malformed JSON, an empty `MessageId` (two different such messages) and `Quantity` = 2 are each dead-lettered after one attempt. None leaves an Inbox row or a Redis effect.
- **Replay.** The dead letter, republished unchanged, applies exactly once.

The fresh-clone run passed all 74 checks (`tests/evidence/20261011T084521Z-failures.log`). A negative control disabled the Inventory empty-`MessageId` check. Five checks then failed and the script exited 1 (saved alongside it as `...-failures-negative.log`), which shows the poison checks have teeth. The Week 9 regressions still pass on the same clone: TP-M01 34/34, TP-M02 44/44, and the workflow and stock-result verifiers.

10.6.3 Reconciliation: stuck orders, a bound and an alert

Retries and the DLQ deal with *messages*. Reconciliation is the safety net for *orders*: it finds an order that is still `PendingStock` long after it should have been answered, and acts in a bounded, visible way. It does not replace the Outbox, retry or the DLQ, and it never changes an order's state to hide a problem (`docs/failure-handling.md`).

Detection and cadence. `ReconciliationWorker` (Order Service) polls on a configurable interval (30 s by default). It looks for orders still `PendingStock` whose request was accepted longer ago than a configurable timeout (also 30 s by default). For each one it inserts a fresh `OrderPlaced` Outbox row, which re-runs the normal pipeline. It acts on an order at most once per timeout window, so a stuck order is not re-triggered on every poll.

The bound and the alert. The attempt count comes from the order's own `OrderPlaced` Outbox rows: the first row is the original delivery, and every later row is one re-trigger. Because the count lives in the database, it survives an Order Service restart. An order is given up only when two things are both true: the count has reached `MaxAttempts` (3), and the last re-trigger has had its full window to work. A given-up order:

- is no longer re-triggered;
- produces one Error log line naming the order, its attempts and its age;
- stays `PendingStock`, because no `Rejected` is invented;
- is counted in three Prometheus metrics: re-triggers, orders given up, and orders currently stuck.

Recovering a known outcome. Before Week 10, a re-trigger often could not recover anything. Suppose Inventory had already published a result and Order Service lost it, for example because it was dead-lettered and the DLQ was purged. Inventory then saw that a result row existed and wrote nothing, so the order stayed stuck while being re-triggered forever. Now a re-triggered `OrderPlaced` for an order whose result was already published re-sends that *same stored result*: the same payload and the same `MessageId`. There is no second Lua deduction (`reserve.lua` answers `DUPLICATE`) and no new row. Order Service's Inbox applies the result once, or ignores it if it was already applied. A reserved order therefore stays reserved and is never rewritten as out of stock.

The operator procedure is in `docs/failure-handling.md`: inspect the order, its Outbox rows and the DLQs; fix the cause; then replay the dead letter unchanged. Stock is never reset and queues are never deleted as a "repair".

Verification (TP-F02). `scripts/verify-reconciliation.ps1` runs all three services with short timeouts (5 s, a 2 s poll, a bound of 3). It covers six cases:

- **A, lost result.** A reserved order's `StockReserved` is dead-lettered and the DLQ purged. Reconciliation re-triggers it, and Inventory re-sends the stored result, whose `PublishedAt` is re-stamped after the purge. The order ends `Completed`, with one deduction, one result row and the *original* result `MessageId`.
- **B, transient outage.** Inventory is stopped, three orders are placed, and only Inventory is restarted. All three drain to `Completed` with one deduction each, and none is `Rejected`.
- **C, permanent fault.** With Inventory kept down, the order is re-triggered exactly three times. The give-up Error is logged exactly once, and the metrics read 3 re-triggers, 1 given up and 1 stuck. The order stays `PendingStock`.
- **D, Order Service restart.** The restarted service resumes from the stored count. It stops at the bound and does not start again from zero.
- **E, no false positive.** A normally completed order is never re-triggered.
- **F, outcome preserved.** When Inventory returns, the given-up orders from C and D complete with exactly one result each. The stuck gauge returns to 0, and the stock is conserved: 20 units minus 7 orders leaves 13.

A fresh-clone run passed all 30 checks (`tests/evidence/20261011T102304Z-reconciliation.log`). A negative control removed the Inventory re-send. Case A then stayed `PendingStock` with its result never re-sent, and the script exited 1.

Confirmed and Processing (after the review). Reconciliation first watched `PendingStock` only, so an order whose Process Worker copy of `StockReserved`, start event or completion was lost stayed `Confirmed` or `Processing` for ever. It now also selects those orders once their confirmation is older than `ProcessingStuckTimeoutSeconds` (300 s by default, never shorter than the `PendingStock` timeout, because the Worker's queue legitimately holds confirmed orders after an overload). The same re-trigger recovers them: Inventory re-sends its stored `StockReserved`, which also reaches the Worker. Case G stops the Worker, lets an order be confirmed, purges the Worker's copy and restarts the Worker; the order ends `Completed` with

one deduction, and all 35 checks pass (`tests/evidence/20261023T142609Z-reconciliation.log`). With the new selection removed, case G fails: the order stays `Confirmed` (`tests/evidence/20261023T143020Z-reconciliation-negative-control.log`).

10.6.4 Supported and unsupported failure modes

Table 10.3 is the failure model this system claims. It uses four statuses:

- **Verified**: a test case with a saved PASS log covers the failure.
- **Designed**: the mechanism exists in code, but the failure was never injected at that exact point.
- **Not supported**: a stated gap, not one hidden by wording.
- **Out of scope**: excluded by the teacher brief.

The same table is kept in `docs/failure-handling.md`. The check script for this chapter (TP-F03) fails if the two disagree, or if a row marked Verified cites a test case that has not passed.

Reading the table. Fifteen failure modes are verified. The crash-point matrix of the study guide is covered as follows:

- a crash before or after the accept commit is F1;
- a crash after the SQL effect but before the ack is F6;
- a failure after Redis but before the SQL result is F4 and F12.

"After the ack, before the effect" cannot happen, because every consumer acks only after its commit. The Week 9 negative control, which acks before processing, fails 24 checks (TP-M01).

Two rows are only designed:

- **F16.** Absorbing a same-`MessageId` copy is verified (TP-M01, TP-M02). A crash at exactly that point was not injected.
- **F17.** Every database fault in these tests was a revoked privilege, not a stopped PostgreSQL server.

Redis data loss (F18) is the main unsupported case. After a Redis loss, the warm-up must be repeated before sales reopen (Week 7). Whether Redis persistence survives a crash without loss has not been verified.

Two rules hold for every row:

Table 10.3. – Failure modes, how each is handled, and the evidence behind it.

#	Failure	Handling	Status	Evidence
F1	Client retries, also after commit but before the HTTP reply	`Idempotency-Key` returns the original order; a changed body gets 409	Verified	TP-A03–A05, TP-A07
F2	Outbox write fails with the order write	One transaction: no order, no event	Verified	TP-O01
F3	Broker down; queue or binding missing; nack; return	Outbox row stays unpublished and is retried; delivered once after recovery	Verified	TP-O02
F4	PostgreSQL write fails after Redis reserved	Retried, then DLQ; replay gets `DUPLICATE` and writes the result once	Verified	TP-I03, TP-F01
F5	Order Service write fails while applying a result	Retried, then DLQ; replay applies it once	Verified	TP-W03, TP-F01
F6	Consumer crash after commit, before ack (all four)	Redelivery hits the Inbox; acked as already processed	Verified	TP-M01
F7	Duplicate deliveries, concurrent or with a new `MessageId`	Inbox, Redis `DUPLICATE`, one result row per order	Verified	TP-M02
F8	Completion arrives before the stock result	Requeued with 2/4/8 s delays; DLQ after 3 requeues	Verified	TP-M02
F9	Transient handler fault	Four attempts (1/2/4 s), then the service's DLQ	Verified	TP-F01
F10	Poison: malformed JSON, empty id, `Quantity` $\neq 1$	The service's DLQ after one attempt, no effect	Verified	TP-F01
F11	Exhausted message	That service's DLQ only; replayed unchanged, applied once	Verified	TP-F01
F12	Published result lost (dead-lettered, DLQ purged)	Reconciliation re-triggers; Inventory re-sends the stored result	Verified	TP-F02
F13	Inventory down for a while	Work drains after restart, one outcome per order	Verified	TP-F02
F14	Inventory down for good	3 re-triggers, one Error alert and metrics; stays `PendingStock`	Verified	TP-F02
F15	Order Service restarts while an order is stuck	Attempt count read from the database, not reset	Verified	TP-F02
F16	Crash after broker confirm, before the Outbox row is marked	Re-published with the same `MessageId`; the consumer's Inbox absorbs it	Designed	–
F17	PostgreSQL server outage	Same path as F4/F5; tests revoke privileges, never stop the server	Designed	–
F18	Redis data loss or restart	Not automatic: re-run warm-up before reopening; persistence not verified	Not supported	–
F19	Overload beyond capacity	Nginx 429, admission 503 (concurrency and Outbox backlog) and bounded queues; backlog drains and the system recovers without a restart (local and EC2, pre-pilot)	Verified	TP-L01, TP-L02, TP-L04
F20	Downstream processing fails (`ProcessingFailed`)	The Process Worker is success-only; no event reaches this state	Out of scope	Brief
F21	Payment failure and compensation	Not part of the system	Out of scope	Brief
F22	Redis Cluster, RabbitMQ Cluster, PostgreSQL replication, multi-region	Single instances on one host	Out of scope	Brief

- No recovery changes an order's state to hide a failure. A stuck order stays `PendingStock` and is surfaced, never rewritten as `Rejected`.
- No recovery resets stock or deletes queues. Recovery is always a replay of the original message, or a re-send of a stored result.

10.7 Process Worker and Observability (Week 11)

Week 11 adds the last component of the pipeline and the instrumentation the experiments will read it through. It also closes the oldest open question in this project: the `Confirmed` → `Processing` → `Completed` leg that `docs/order-state-machine.md` had refused to specify since Week 1.

10.7.1 The Process Worker, and what it settled about the state machine

`FlashSale.ProcessWorker` is a third service: it consumes `StockReserved`, waits a deterministic delay, and publishes `OrderProcessed`. Per Proposal §3 it simulates downstream processing only – there is no payment step, no failure injection and no compensation logic, all three being excluded by `docs/00-scope-lock.md`. The delay is deterministic rather than randomised on purpose: with a fixed service time, any variance Week 13/14 measure in end-to-end latency comes from queueing and contention in the architecture, not from the simulator adding noise of its own.

It consumes from its *own* queue (`ProcessWorker.StockReserved`), bound to the same exchange with the same `StockReserved` routing key as Order Service's queue. A direct exchange copies the message to every bound queue, so both services react to the same event independently. Sharing one queue would have made them compete for messages, and roughly half of all orders would never have been processed at all.

The six named states in the code. An earlier Week 11 draft removed `Processing` and `ProcessingFailed`, because Order Service only learned two facts after placement. The project then adopted the teacher's six named states (Section 9.5), and the code now implements them.

- **A real trigger for `Processing`.** When the Process Worker begins work on a reserved order, it publishes `OrderProcessingStarted` (routed, with

publisher confirms). Its `MessageId` is derived from the order id: a name-based UUID that differs from the `OrderProcessed` id (`MessageId = OrderId`). This keeps the two events apart in the Inbox, and a replay still regenerates the same id.

- **A consumer like the others.** Order Service consumes the event on its own queue, with the Inbox, the 1/2/4 s retry ladder, validation and a DLQ.
- **`ProcessingFailed` is declared but unreachable.** The Worker is success-only by design, so no event and no code path sets it. A unit test and a source scan prove this.

History, and events that arrive out of order. Every transition writes a row to `order_status_history`, in the same transaction as the state change. The order's creation writes the first row. A duplicate or stale event is a no-op and writes no row. The API's polling response returns the history.

The Worker consumes `StockReserved` in parallel with Order Service, so its events often arrive before the order is `Confirmed`. They are then requeued on bounded delayed retry queues, never dropped. If the completion arrives before the start event, Order Service applies `Confirmed` → `Processing` → `Completed` in one transaction. It marks the `Processing` row as implied, and the start event later becomes a no-op.

The review found two defects in the first version:

- The implied row took the Worker's start time. That time can be earlier than the moment Order Service recorded `Confirmed`, so the history sorted `Processing` before `Confirmed`.
- The two rows written together could be inserted in either order.

The fix has three parts:

- The history is read by the identity column `Seq`, not by time.
- The implied row's time is the later of the Worker's start time and the confirmation time.
- The implied path saves the `Processing` row and the `Completed` row in two steps inside one transaction, so their order is fixed and the change stays atomic.

A completion without a start time, such as one published by an older Worker, uses the time it is applied, instead of the year 0001.

Verification (TP-P02). The unit tests (90) cover:

- the transition table;
- every 5-event sequence over the three Worker-facing events, 150 in total, each ending `Completed` with the four-step history;
- the clamped, kept and missing start times;
- the `Seq` order.

`scripts/verify-processing.ps1` runs all three services and covers these cases:

- normal orders by either legal path;
- a deterministic direct path, with the derived start id;
- duplicates, which change nothing;
- the late start event, which gives an implied row clamped to the confirmation time, then a no-op;
- 10 orders on the implied path, all in `Seq` order;
- Worker events while `PendingStock`, which are requeued and then applied;
- no `ProcessingFailed` anywhere;
- a poison start event, which goes to its DLQ.

The fresh-clone run passed all 41 checks (`tests/evidence/20261013T145229Z-processing.log`). A negative control that writes the implied row as not implied fails two checks, and the script exits 1.

Idempotency with no database. The Process Worker is the only service here with no persistent store, and deliberately so: it holds no business state, so there is no local write that a publish could disagree with, and therefore no dual-write problem and no need for an Outbox – the reasoning that forced Inventory Service to add one in Week 8 simply does not apply. Step 11.1's requirement that it "use `order_id` as its own idempotency key" is met by deriving the published message's `MessageId` from the order id (they are set equal) instead of minting a fresh one per hop as every other event does. A redelivered `StockReserved` therefore regenerates an `OrderProcessed` with the same `MessageId`, which Order Service's Week 9 Inbox recognises and drops (only its start time can differ): the idempotency key travels in the message rather than living in a store.

A restart defect found and fixed. The code check found that a *graceful* shutdown of the Worker (Ctrl+C, or SIGTERM from `docker stop`) dead-lettered accepted work. Stopping the host cancels the processing delay. The cancellation was caught as a failed attempt, the retry wait then threw again, and the message was nacked to the DLQ. The broker had already pushed every queued message to the Worker, so each of those failed the same way. A throwaway prototype with 10 queued orders dead-lettered 9 of them on one Ctrl+C. Now a cancellation caused by shutdown is not a failed attempt. The message is left unacknowledged, so the closing channel returns it to the queue, and one Information line names the order. Real failures still take the 1/2/4 s ladder to the DLQ.

Verification (TP-P01). `scripts/verify-worker.ps1` runs all three services on a throwaway broker and Redis (`docs/process-worker.md`). It covers seven cases:

- **Deterministic success.** With a 500 ms delay, 5 orders complete. Each Worker duration is at least 500 ms; the measured values were 517–643 ms. Each applied completion has `MessageId = OrderId`.
- **Replay with the same identity, a duplicate with a new `MessageId`, and 5 concurrent copies.** The Worker repeats the work, and its attempt counter rises by the number of copies. Order Service still has one completion Inbox row, an unchanged `CompletedAt`, and no dead letter. Attempts and applied results are counted separately.
- **Graceful restart (a real Ctrl+C) and a crash.** Each restart happens with 5 orders in flight. The DLQ stays empty and the Worker logs no DLQ routing or error. After the restart, all 5 orders complete with one applied completion each.
- **Reordering.** A completion that arrives while the order is still `PendingStock` is requeued, not dead-lettered. It is applied exactly once after the stock result, with `ConfirmedOrRejectedAt` $\leq$ `CompletedAt`. A late stock result after completion is a no-op.

The fresh-clone run passed all 51 checks (`tests/evidence/20261011T160720Z-worker.log`). With the shutdown fix reverted, the Worker logged 5 DLQ routings and 5 errors during shutdown, and the script exited 1. The dead-letter count alone was not a reliable signal: the reverted Worker's rejections sometimes reached the broker after the channel had closed, and those messages were requeued anyway. So the check reads what the Worker did, not only where the messages ended up.

A test defect found in Week 11. A later fresh-clone run failed the graceful-restart case although the Worker was correct. The cause was Windows: a script started from Git Bash or a background task runs with Ctrl+C ignored, and the Worker inherited that. The test's Ctrl+C call then succeeded, but no Ctrl+C reached the Worker. The same run exposed a second defect: the check for the Worker's shutdown line searched the whole log file, so a line from an earlier run made it pass. Now the script turns Ctrl+C back on before it starts the Worker. It proves delivery from the Worker's own "Application is shutting down" line, and it reads only the log lines written after the Ctrl+C. Started from Git Bash, the fixed script passed all 51 checks twice, and then again from a fresh clone on a fresh database (`tests/evidence/20261014T090016Z-worker.log`). Without the Ctrl+C fix it fails the delivery check, the exit check and the shutdown-line check, and exits 1.

10.7.2 Correlation IDs across three services (Step 11.2)

Each event now carries a `CorrelationId`, copied unchanged at every hop. That invariance distinguishes it from the two identifiers already in the contract: `MessageId` is regenerated at every hop (each consumer needs its own Inbox key), and `OrderId` only exists once an order does, whereas a correlation id identifies the *request*.

Order Service does not mint the value. ABP's own `app.UseCorrelationId()` middleware was already populating `ICorrelationIdProvider` from the caller's `X-Correlation-Id` header (generating one per request when absent), and `app.UseAbpSerilogEnrichers()` was already stamping that value onto every log line written during the request. Generating a second id in the controller would have made the HTTP log lines and the order row disagree about what the request is called – the precise failure this step exists to prevent – so the controller takes ABP's value and carries it onto the order and every event that follows. Across the message hops, where no HTTP middleware exists, each consumer re-establishes it with `LogContext.PushProperty`, which flows through the awaits into everything that logs downstream of it.

Every log line also carries a `Service` property, so one search over the three JSON logs shows which service wrote each line. The Process Worker's `Received` line now names the incoming `MessageId`, so the `StockReserved` it consumed can be matched to the one Order Service consumed.

Verification (TP-P03). `scripts/trace-order.ps1` places three orders, each with its own `X-Correlation-Id`:

- one that completes;
- one that Inventory rejects;
- one that completes after a recovered fault, with Order Service's Inbox `INSERT` grant revoked for about 3 s.

For each order it searches the three JSON logs for that one id and checks that:

- every expected hop is present, each line tagged with its service;
- the `MessageIds` link up: the id in each Outbox row is the id the next consumer logs;
- every stage timestamp is present and ordered (acceptance $\leq$ publication, and acceptance $\leq$ decision $\leq$ applied $\leq$ completion);
- the history, read by `Seq`, and the API's polling response agree.

The completed order from the fresh-clone run reads (ids shortened):

```
OrderService     16:33:12.38  Executing endpoint
↪  'OrdersController.CreateOrderAsync'
InventoryService 16:33:14.81  Received OrderPlaced 6eeb0972-... for order
↪  32fe1f82-...
InventoryService 16:33:15.19  OrderPlaced 32fe1f82-... for product tr-163219 ->
↪  Reserved
OrderService     16:33:16.49  Received StockReserved 631ba675-... for order
↪  32fe1f82-...
OrderService     16:33:16.60  Order 32fe1f82-... -> Confirmed
ProcessWorker    16:33:16.84  Received StockReserved 631ba675-... for order
↪  32fe1f82-...; starting downstream processing
OrderService     16:33:16.96  Received OrderProcessingStarted 24eaf83a-... for
↪  order 32fe1f82-...
OrderService     16:33:16.98  Order 32fe1f82-... -> Processing
OrderService     16:33:17.28  Received OrderProcessed 32fe1f82-... for order
↪  32fe1f82-...
ProcessWorker    16:33:17.30  Order 32fe1f82-... processed in 434ms; published
↪  OrderProcessed
OrderService     16:33:17.30  Order 32fe1f82-... -> Completed
```

Three details in this trace are worth explaining:

- Order Service and the Worker log the same `StockReserved` id (`631ba675`), because the exchange copies one message to both queues.
- The `OrderProcessed` id is the order id, which is the idempotency design above.

- Order Service logs receiving `OrderProcessed` 20 ms before the Worker logs publishing it. This is not a clock error. The Worker writes its line after the broker's confirm returns, but the broker delivers the message as soon as it routes it. For the same reason, an Outbox row is marked published only after the confirm. In the fault trace, taken while `PublishedAt` still held the confirm time, the `OrderPlaced` row's `PublishedAt` came 1.1 ms after Inventory's decision. It now holds the send time (Section 10.11). The script prints such a case as an anomaly and does not count it as a failure (`docs/metrics.md`).

In the fault trace, both of the Worker's events arrived while the order was still `PendingStock`. They were requeued, and the order ended `Completed` with the full four-step history once the grant came back. The fresh-clone run passed all 28 checks (`tests/evidence/20261013T163219Z-trace.log`). A negative control that drops the `MessageId` from the Worker's `Received` line fails four checks: the missing Worker hop and the broken `MessageId` link, in both the success trace and the fault trace. The script exits 1.

10.7.3 Structured logging and metrics (Step 11.3)

Logging. Serilog was already configured in all services with `Enrich.FromLogContext()`; what changed is the file sink's format, from plain text to one JSON object per line (`CompactJsonFormatter`). Structure is what makes the correlation id above actually *queryable* – Week 13/14 need to filter thousands of lines by a field and read timestamps back as values, not regex them out of a rendered string. The console sink deliberately stays human-readable: it is watched live by a person, not parsed.

Metrics. Prometheus and Grafana were added to `infra/docker-compose.yml`, with the Grafana datasource provisioned from a file so a fresh `docker compose up` returns a working stack (the same reproducibility reasoning behind `reset-and-seed.ps1`). All three services expose `/metrics` via `prometheus-net`. Because the services run on the host rather than in Compose, Prometheus scrapes `host.docker.internal`; the two ABP services are scraped over HTTPS with `insecure_skip_verify`, since they run on the ASP.NET Core development certificate. The scrape interval is 5s rather than Prometheus's 15s default: a load run lasting minutes would otherwise produce too few points to read a ramp or a recovery from.

`docs/metrics.md` is the metric dictionary. For every metric in the teacher's brief and in the experiment plan, it gives:

- the definition, with its boundaries and denominator;
- the one source the value is read from;
- how the value is extracted;
- whether that source exists now or comes in Week 12–14.

Table 10.4 maps each metric the brief requires to its source and its status today. "Collected" means the run-time collector records the value during a run. "Extracted" means the extraction script computes it after the run, from the JTL, the database or the collected samples. Both scripts are described below. Every collected value is also summarised by the extraction.

The exported metrics fall into five groups:

- **Business results.** `flashsale_orders_accepted_total` counts orders, and only after the commit, so it can be compared with JMeter's request count. `flashsale_orders_outcome_total{outcome}` counts state changes that were actually applied, never an idempotent no-op, so it cannot drift from the database under redelivery. Inventory's `flashsale_stock_decisions_total{outcome}` has the values `reserved`, `rejected`, `not_open` and `duplicate`. Each counts a business result, never an attempt.
- **Work attempts, kept apart from results.** The Worker's `flashsale_orders_processed_total` and `flashsale_process_worker_duration_seconds` count every processing run, including replays. A replay therefore raises the attempt count but not the completion count.
- **Reliability.** All three consumers export `flashsale_consumer_retries_total` and `flashsale_consumer_dead_lettered_total`, both labelled by consumer. The dead-letter counter also has a reason: `exhausted`, `invalid` or `conflict`. Order and Inventory add `flashsale_consumer_duplicates_total{consumer}`, and Order adds `flashsale_consumer_requeued_early_total{consumer}`. The reconciliation metrics come from Week 10.
- **Backlog.** Order and Inventory export `flashsale_outbox_unpublished` and `flashsale_outbox_oldest_unpublished_age_seconds`, refreshed on every publisher poll.
- **Resources.** The `prometheus-net` defaults give each process's CPU seconds and working set.

Labels hold only a few fixed values (consumer, outcome, reason). Order ids and message ids stay in logs and database rows. A label per order would give one

Table 10.4. – Metric definitions: source and status of every metric in the brief. Prometheus names omit the `flashsale_` prefix. Full definitions, denominators and extraction are in `docs/metrics.md`.

Metric	Source	Status
API latency p50/p95/p99	JMeter JTL (`elapsed`), submissions only	Extracted
Stock-decision latency	Request start to Inventory's result `CreatedAt` (C2–C4); JTL (C1)	Extracted
End-to-end completion latency	Request start to `CompletedAt`	Extracted
Incoming, accepted, completed throughput	JTL; `orders_accepted_total`; `orders_outcome_total{Completed}`	Extracted
Rejected request rate (overload)	JTL 429/503	Extracted; the limits that cause it come in Week 12
Internal error rate	JTL 5xx	Extracted
Reservation success / rejection rate	`stock_decisions_total{outcome}`; Inventory results	Extracted
Queue depth	RabbitMQ management API	Collected
Oldest message age	Each message's publish timestamp, read from the queue's head message	Collected
Backlog drain time	Queue depth, Outbox gauges, pending orders	Extracted
Recovery time	The time series above	Extraction ready; thresholds from the pilot
Retry and duplicate count	`consumer_retries_total`, `consumer_duplicates_total`	Extracted
DLQ message count	`consumer_dead_lettered_total`; DLQ depth	Extracted
Pending order count	Each order's own timestamps	Extracted
CPU, RAM, DB connections	`process_*` per service; `pg_stat_activity`; host counters	Collected (Linux path checked in a container and on EC2 in Week 12, where the Process Worker's endpoint was not reachable yet)
Redis latency	Redis `INFO commandstats` (Lua calls)	Collected
RabbitMQ publish, deliver, ack rates	Management API `message_stats`	Collected

series per order and overload Prometheus under the very load being measured. Every labelled series is created at start-up, so a rate over a run that saw no retries reads 0 and is never missing.

Verification (TP-P04). `scripts/verify-metrics.ps1` reads each service's `/metrics` before and after eight cases. It checks that each case moved the expected series by the expected amount:

- three normal orders: accepted, completed, reserved and Worker attempts each $+3$;
- a stock rejection: both rejection counters $+1$;
- a duplicate `StockReserved`: duplicates $+1$, and the confirmed count unchanged;
- a transient Inbox fault: retries ≥ 1, no new dead letter, and the order completes;
- a poison message: `dead_lettered{invalid}` $+1$;
- an early completion: requeued-early ≥ 1;
- a paused broker: the Outbox gauges rise while it is paused and return to 0 afterwards;
- a Worker replay: attempts $+1$, but still one completion Inbox row.

It then checks that each service exports every metric the dictionary lists for it (13, 8 and 7), and that no label value is an id. The fresh-clone run passed all 17 checks (`tests/evidence/20261013T163454Z-metrics.log`). A negative control that stops counting duplicate stock results fails case C (duplicates $+0$), and the script exits 1.

Run-time collection and extraction (W11 milestone). Several brief metrics exist only while a run is in progress, and Prometheus does not hold them: queue depth, the age of the oldest queued message, broker rates, Redis latency, database connections and host load. Three additions make every metric recordable and computable before the pilot:

- **Message timestamps.** Every publish (both Outbox publishers, both Worker publishers and both delayed requeues) now stamps the AMQP `timestamp` property. RabbitMQ then reports the oldest queued message's age per queue, at the resolution of its statistics interval (5 s by default). A unit test scans

the source, so a future publisher cannot forget the stamp. TP-P06 checks all six paths on a live broker.

- **A run-time collector** (`scripts/collect-run-metrics.ps1`). Every 5 s it appends the queue counts and ages, the broker rates, the Redis Lua call totals, the connections per service account, every service metric, and host CPU and memory to one long-format CSV. A source that fails is recorded as `unavailable`, never as 0, and the other sources keep sampling. One sample takes about 4 s on the local host, so 5 s is the shortest usable interval. TP-P07 checks it with Inventory stopped (the backlog grows) and Redis stopped (its rows become `unavailable`).

- **An extraction script** (`scripts/extract-run-metrics.ps1`). It turns one run's JTL, database rows and collected samples into the numbers the Results tables ask for. Percentiles use the nearest-rank method. Negative and missing stage intervals are counted, never hidden. Counter increases survive a restart, because each series is handled on its own. Drain time waits for the queues, the Outbox gauges and the run's own pending orders to reach 0. Recovery time stays `unavailable` until the pilot sets its thresholds. TP-P08 checks it against hand-computed answers, including a counter reset, a negative interval and a late completion, and against a real run.

Writing these checks found four defects that a single test run would have hidden: a seven-hour shift in every time window (this machine is UTC+7, the EC2 host is not), a false 0 from a broker statistic that had not arrived yet, a counter sum that over-counted after one service restarted, and a pending count that included older orders and so could never drain. The time window, the counter reset and the older orders each have a known-answer check whose negative control fails as designed. The broker statistic is now recorded as `unavailable` until it arrives. That path is shown in the code and the test design, not by a separate test. From a fresh clone on a fresh database, TP-P06, TP-P07 and TP-P08 passed 40, 15 and 35 checks, and every earlier runtime verifier still passed (`tests/evidence/20261014T140055Z-message-timestamps.log`, `20261014T140223Z-run-collector.log`, `20261014T140415Z-extract.log`).

Latency percentiles are deliberately *not* computed in-process. JMeter measures client-observed latency directly, and end-to-end pipeline latency is reconstructed from the per-order timestamp schema below. A Prometheus histogram of the same quantity would be a third, bucketed copy that disagrees with both at the tail – which is exactly where this project's research question lives.

Table 10.5. – Stage timestamp schema. "Order" is `order_service`, "Inventory" is `inventory_service`.

Stage	Column	Moment recorded
Request accepted	Order `orders.RequestAcceptedAt`	When Order Service builds the order, just *before* the order and its Outbox row commit.
Event published	Order `outbox_events.PublishedAt` (`OrderPlaced` row)	When the publisher marks the row, *after* the broker's routed confirm.
Stock decided	Inventory `outbox_events.CreatedAt` (the result row)	When Inventory writes the result, in the same transaction as its Inbox row, right after the Lua decision.
Decision applied	Order `orders.ConfirmedOrRejectedAt`	When Order Service applies `StockReserved` or `StockRejected`.
Processing started	Order `order_status_history.At` (`Confirmed` → `Processing`)	When Order Service applies `OrderProcessingStarted`. For an implied row, the later of the Worker's start and the confirmation.
Completed	Order `orders.CompletedAt`	When Order Service applies `OrderProcessed`.

Timestamp schema (Proposal §4d). Table 10.5 lists every stage time the experiments use, with the column it is read from and the exact moment it records. The columns are EF Core columns in each service's own schema, so their names are PascalCase. All values are UTC, at microsecond precision, and come from one host's clock, because the experiments run every backend service on one EC2 instance. `docs/metrics.md` is the full version.

Two of these boundaries are not where a reader might assume, and both are declared rather than corrected:

- `RequestAcceptedAt` is set before the commit, so the order only becomes durable slightly later.
- `PublishedAt` is the time the confirmed publish was *sent*. Until the review after Week 12 it was the time the confirm came back; since the broker delivers before it confirms, 714 of 773 publish-to-decision intervals in the first TP-L04 run were negative. With the send time there were none in the following runs. A negative interval would still be reported as an anomaly, never clamped to zero, and a missing value is reported as missing, never as 0.

Publication time is read from the order's own Outbox row, joined by order id, and is not copied onto `orders`. Copying it would add a second write to the publish path of the very system whose throughput is being measured.

Replays after completion. When a `StockReserved` for an order that has already completed is delivered again, the two consumers behave differently, and both are correct:

- The Process Worker has no store, so it does the work again and republishes `OrderProcessed` with the same message id. Order Service's Inbox recognises that id, so the completion is not applied twice. TP-P04 case H measures this: the Worker's attempt counter rises by one, and the order still has one completion Inbox row.
- Order Service's own consumer treats a copy with the same message id as a duplicate (TP-P04 case C: duplicates $+1$, confirmed count unchanged). A stock result with a new message id, or one arriving after `Completed`, is a no-op, because the order has already moved past `Confirmed` (TP-M02). Since Week 9 neither case is dead-lettered. A dead letter is kept for poison messages and exhausted retries, and each service's dead letters stay in its own DLQ (TP-F01).

10.8 Load Shedding

C3 adds three limits in front of the parts that cost real work, as brief §4 asks: rate limiting at Nginx, concurrency limiting in the application and bounded queues in RabbitMQ. A new request is refused early with a response code that means "too busy". Every limit is off by default, so C2 behaves exactly as before until a C3 configuration switches the limits on. The design is in `docs/load-shedding.md`; Figure 10.1 shows where each layer refuses work.

Nginx. `infra/nginx.conf` is a reverse proxy to Order Service. Its upstream, rate and burst come from the environment. Only `POST /api/orders` is limited, in any letter case and with or without a trailing slash: a `map` on the method and path gives every other request an empty key, which `limit_req` ignores, so status polling is never refused. The key is the client IP, which acts as one global bucket because JMeter runs from one machine.

Admission. Order Service holds a zero-wait semaphore whose size is `Admission:MaxConcurrentNewOrders`. The check sits after the idempotency lookup and

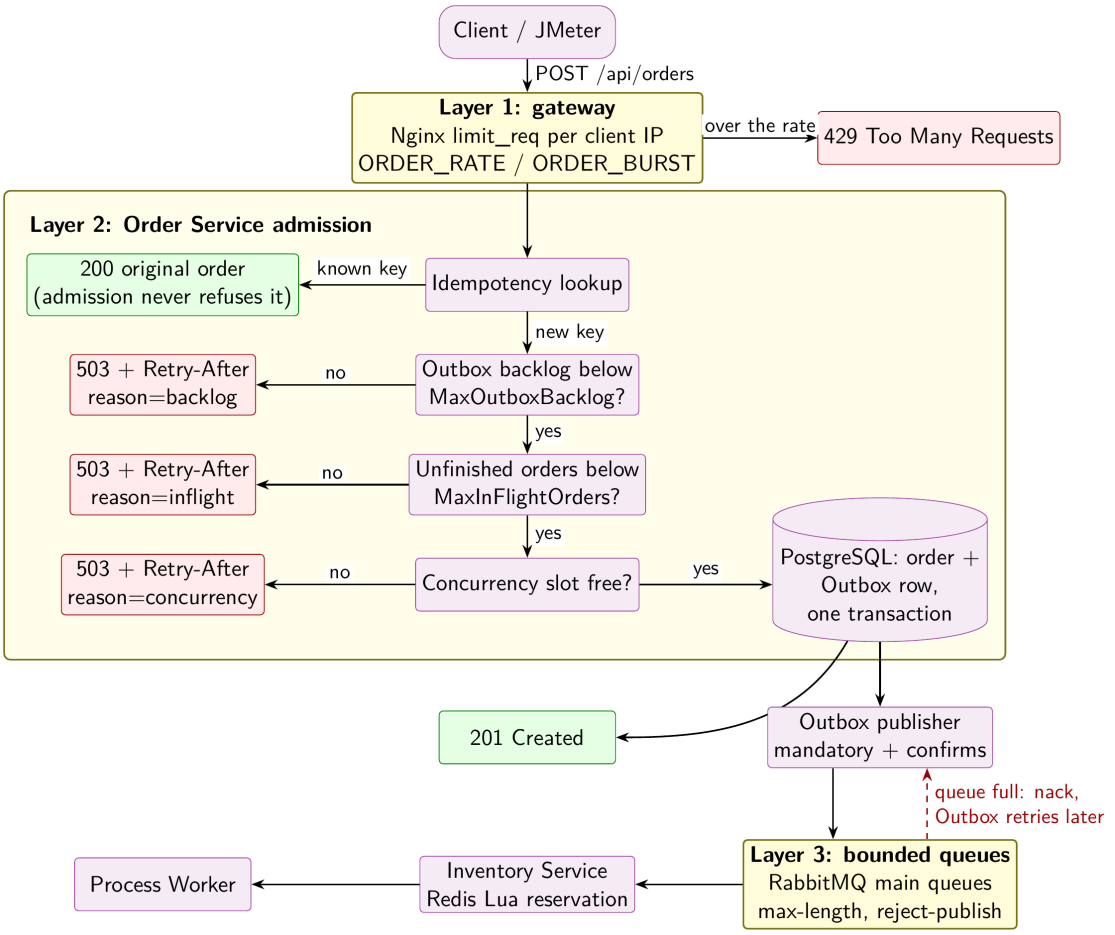


Figure 10.1. – The three load-shedding layers of C3. Order Service answers a retry of an accepted order before its own limits, but Nginx can still refuse the retry with 429, so clients retry with back-off. A refused request stops before the database write, and a full queue only delays publication: the Outbox keeps the event and retries it.

Layer	What is refused	Client sees	Counted in
Nginx rate limit	`POST /api/orders` above `ORDER_RATE`	429	Nginx access log
Order Service admission	A new order while the concurrency gate is full	503, `Retry-After: 1`	`flashsale_admission_rejected_total`
Outbox backlog	A new order while the unpublished Outbox is at `MaxOutboxBacklog`	503, `Retry-After: 1`	`flashsale_admission_rejected_total (reason=backlog)`
In-flight orders	A new order while `MaxInFlightOrders` orders are not yet `Completed` or `Rejected`	503, `Retry-After: 1`	`flashsale_admission_rejected_total (reason=inflight)`
Bounded queue	A publish to a main queue at `max-length`	Nothing (order already 201)	`flashsale_outbox_unpublished`

Table 10.6. – The load-shedding layers of C3 (the in-flight bound was added after the review, Section 10.11).

before any database write. A retry of an accepted order therefore always gets its original order back, even when the gate is full, and a refused order leaves no row and no Outbox event. The slot is released in a `finally` block, so an exception or a cancelled request cannot leak it.

Backlog bound. A bounded queue alone does not bound admission: when a queue is full or the broker is down, accepted orders wait in the Outbox, which could grow without limit. Order Service therefore also refuses a new order (503, `reason=backlog`) when the unpublished Outbox has reached `Admission: MaxOutboxBacklog`. The check runs after the idempotency lookup and before the concurrency gate. The backlog is not counted per request: the Outbox publisher measures it on every poll, and each order accepted since that measurement is added at once. The bound therefore holds even while a poll is stuck retrying an unavailable broker. It can be overshot only by requests that pass the check at the same moment, which the concurrency gate caps.

Bounded queues. A RabbitMQ policy with a `max-length` and the overflow mode `reject-publish` bounds the six main input queues and never the retry or dead-letter queues. It is set and removed at run time by `scripts/set-queue-bounds.ps1`, with no change to any queue declaration. A publish to a full queue is nacked; the publisher's mandatory publish with confirms (Section 10.4.4) leaves the Outbox row unpublished, and it is retried until the queue has room. Every

consumer takes at most 20 unacknowledged messages (a prefetch, added after the review), so the bound covers real work; without it the Process Worker took its whole backlog as unacknowledged messages, which `max-length` does not count.

In-flight bound (after the review). The Outbox bound protects only the first hop. Order Service therefore also refuses a new order (503, `reason=inflight`) while `Admission:MaxInFlightOrders` orders are still unfinished anywhere in the pipeline. The count is estimated the same way as the backlog (a measurement about once a second, plus every order accepted since) and is only measured while the bound is on, so C2 pays nothing. Section 10.11 explains why it was needed.

Verification. Test case TP-L01 (`scripts/verify-load-shedding.ps1`) passed 18 of 18 checks on local services:

- **Gateway**, at 5 requests per second: 40 rapid POSTs gave 4 accepted and 36 refused with 429. Ten more POSTs each to `/api/orders/` and `/API/orders` were all refused, since they share the same limit. No refused key has an order row, and 15 polls through Nginx were never refused.
- **Admission**, with 2 slots held for 1.5 s: 10 parallel POSTs gave 2 accepted and 8 refused with 503 and `Retry-After`. The rejection counter rose by exactly 8, no refused key has an order row, and a replay of an accepted key returned 200 with the original order while the gate was full.
- **Bounded queue**, at a length of 3 with Inventory stopped: all 8 orders were accepted, `OrderPlaced` held 3, and the Outbox held the other 5. After Inventory started, the Outbox drained to 0, every order completed, and stock fell by exactly 8.
- **C2 unchanged**: with every limit off, 8 orders were accepted and completed.

Each limit was then removed in turn (rate 10 000 per second, admission off, no queue policy), and each removal made its case fail, so the checks detect a missing limit. Run against the earlier Nginx configuration, which limited only the exact path, the trailing-slash and upper-case check fails, as it should. The unit tests (91) and the earlier runtime test cases still pass.

Test case TP-L02 (the same script, 29 of 29 checks in total) covers the backlog bound and the accounting. With the broker stopped and a bound of 5, the first 5 new orders were accepted and the next 5 were refused with 503 and `Retry-After`. The `backlog` counter rose by exactly 5, and no refused key left an order row, an Outbox row or any change to Redis stock or the reservation set. A retry of an accepted key still returned its original order. Once the broker returned, the Outbox

drained to 0, all 5 orders completed, stock fell by exactly 5, and new orders were admitted again. The metrics collector recorded the backlog rising, the `backlog` rejections and the queue depth. The validator then passed the run with `-Config C3` (Section 10.4.3). With the bound switched off, all 10 orders were accepted and the case fails, as it should.

10.8.1 Overload, drain and recovery (preliminary, local)

Scenario. Test case TP-L04 drives the full C3 stack through Nginx with a three-phase JMeter plan (`tests/jmeter/overload-recovery.jmx`): 30 s at 3 orders/s (baseline), 60 s at 60 orders/s (overload), and 60 s back at 3 orders/s (recovery). Every limit is on: Nginx at 20 requests/s with a burst of 1, admission at 8 concurrent new orders and an Outbox backlog of 20, and a queue bound of 5. These values are local and pre-pilot; the Week 13 pilot fixes the official ones. The baseline rate is deliberately below the slowest stage: the Process Worker spends a fixed 200 ms per order, and the run measured it at about 3.7 orders/s. At 5 orders/s, an earlier run showed its in-flight backlog still growing during "recovery", so that phase could not show a return to a stable state.

What is measured. The Week 11 collector samples the queues, the Outbox and the services during the run, and the extraction script turns them into numbers. The *backlog drain after overload* is the time from the end of the overload phase until all queued work (both services' unpublished Outbox rows and the main queues' ready and unacknowledged messages) is back at its baseline level, while recovery load continues. The *final drain* is the time from the last submission until every queue, the Outbox and every pending order are empty. The *recovery time* runs from the start of the recovery phase to the first 10 s window whose requests are back in range: no refusals, no errors, p95 at most 1 s, and at least one accepted order per second. The review after Week 12 tightened both the backlog and the recovery definition; the earlier ones missed the Worker's backlog and could count a phase that refused everything as recovered.

Preliminary observations (local PC, one run each). Table 10.7 compares three local runs and the EC2 run. The first local run (22 October) passed every check of the time: the 2,390 refusals logged by Nginx and the 832 counted by the admission metric matched JMeter exactly, all 773 accepted orders completed with one unit of stock each, and no service restarted. Its 3.9 s backlog drain, however, missed the Process Worker's backlog. Once every consumer set a prefetch and all queued work was counted (second run), the real picture showed: Order Service

accepted about 13 orders/s during the overload while the Worker finishes about 3.7/s, orders took about 40 s from acceptance to completion, and the backlog took 105 s to clear, failing the 60 s target. The Worker's bounded queue also held back Inventory's result to Order Service, so orders sat in `PendingStock` for about 32 s. With the in-flight bound at the database baseline plus 30 (third run), Order Service refuses what the Worker cannot finish (Figure 10.2): 548 orders were accepted, each completed in about 6 s (p95 6.8 s), the backlog cleared in 6.9 s, and all 45 checks passed (`tests/evidence/20261023T144923Z-load-shedding.log`, with the run's raw JTL, samples and summary saved beside it). With the in-flight check removed from the code, its case fails and the backlog drain is 105 s again (`tests/evidence/20261023T145745Z-load-shedding-negative-inflight-off.log`). In every run the refusals matched JMeter, the Nginx log and the admission metric exactly, no sampled refused key left an order, and every accepted order completed with exactly one unit of stock. With every limit switched off, 3,747 orders got in and the backlog had not drained when the case gave up, so the test detects missing limits.

Repeat on the EC2 deployment (preliminary, one run, earlier code). The same plan and limits ran on the EC2 stack deployed with the Week 12 Terraform and Compose files, with JMeter on its own `c7i-flex.large` host and the backend on an `m7i-flex.large`. Of 4,029 order submissions, Nginx refused 2,411 with 429 and the admission limits 720 with 503; both counts matched JMeter exactly, and ten sampled refused keys of each kind had no order row. All 898 accepted orders completed, stock fell from 10,000 to 9,102 and the validator passed C3. Everything had drained 48 s after the last submission, the recovery phase had no errors (p95 19 ms), and no container restarted. Every admission refusal came from the Outbox-backlog bound, including a few during the slow phases, so that threshold, like the others, must be set at the Week 13 pilot. This run used the code from before the review, and its raw JTL and samples were not copied back before the stack was destroyed, so it cannot be re-measured with the new definitions. It was not repeated on the new code; its numbers describe the earlier code only.

10.9 Deployment on AWS EC2

The brief requires the system under test on one EC2 instance with JMeter on a separate machine. Figure 10.3 shows the deployment that Terraform creates (`infra/terraform/`) and the runbook `docs/deployment.md` drives.

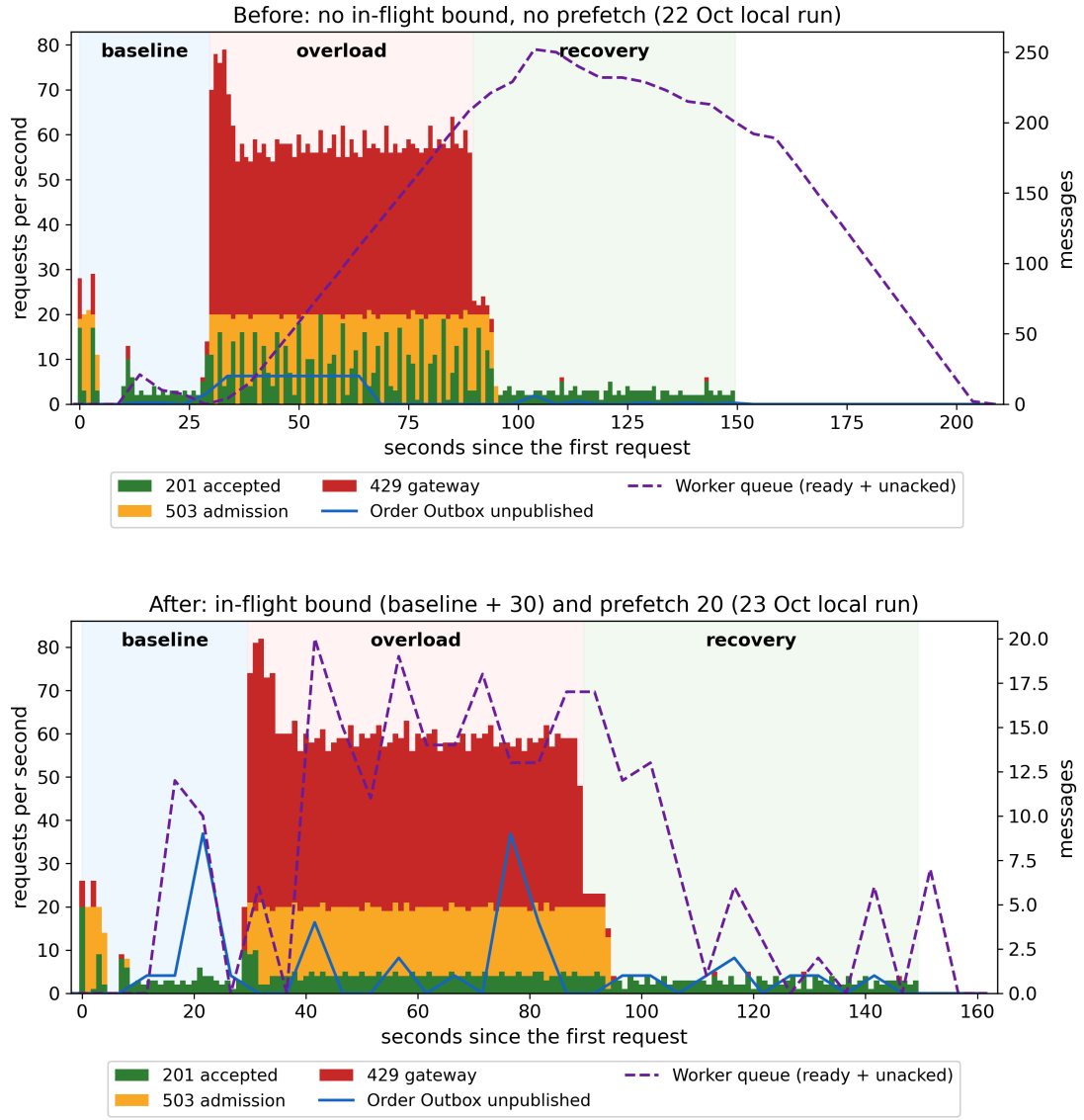


Figure 10.2. – TP-L04, local runs before (top) and after (bottom) the review fixes. Bars: requests per second by outcome (accepted, admission 503, gateway 429). Lines: the Order Outbox backlog and the Process Worker's queue (ready + unacknowledged; note the different scales). Before, the Worker's queue grew to about 250 and drained only after the load stopped; after, it stays below 20 and the run settles within seconds. Data: `tests/evidence/20261022T152931Z-overload-*` and `tests/evidence/20261023T144923Z-overload-*`; chart: `report/figures/plot_week12.py`.

TP-L04 run (pre-pilot)	Local 22 Oct (before)	Local 23 Oct (+ prefetch)	Local 23 Oct (+ in-flight)	EC2 22 Oct (before)
Order submissions (3 phases)	3,995	4,027	4,084	4,029
Refused by Nginx (429)	2,390	2,403	2,464	2,411
Refused by admission (503)	832	816	1,072	720
Accepted and completed	773	808	548	898
Lost orders / extra stock	0 / 0	0 / 0	0 / 0	0 / 0
Accepted to Completed p50/p95	38 / 64 s	40 / 66 s	6.1 / 6.8 s	–
Backlog drain after overload	3.9 s[a]	105 s	6.9 s	2.4 s[a]
Final drain after last request	59 s	55 s	7.1 s	48 s
Recovery time	7 s	5 s	7 s	–
Recovery-phase p95 latency	63 ms	55 ms	34 ms	19 ms
Service restarts	0	0	0	0

Table 10.7. – The same overload plan, one run each. Counts match JMeter, the Nginx log and the admission metric in every run. [a]The earlier, narrower measure that missed messages already delivered to the Worker; not comparable with the other two. The 22 October local run was re-extracted with the current extractor; the EC2 run could not be.

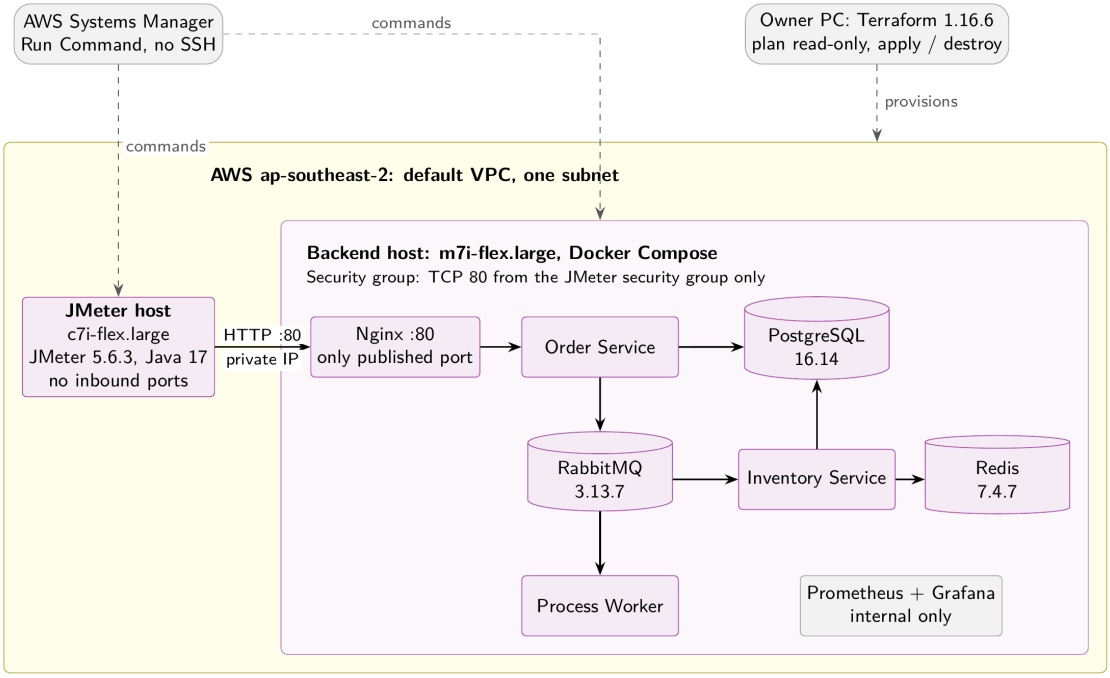


Figure 10.3. – EC2 deployment (Week 12). Both hosts sit in one subnet of the account's Region. The backend runs every service with Docker Compose, and only Nginx publishes a port; its security group admits TCP 80 from the JMeter host's group and nothing else. Both hosts are reached through AWS Systems Manager, so neither opens SSH.

Design. Two instances, a backend (`m7i-flex.large`) and a JMeter host (`c7i-flex.large`), in `ap-southeast-2`. The account's AWS plan fixes that Region and these instance types. The design creates no NAT gateway, no Elastic IP, no load balancer and no managed database. Instances require IMDSv2, use encrypted gp3 disks that are deleted with them, and terminate themselves after four hours if a session is forgotten. Every container image is pinned to the exact version the local tests ran on. The database passwords on EC2 are generated on the host and never leave it.

Who does what. The owner runs `terraform apply` and `terraform destroy`. The assistant only reads the account, using a read-only identity for `terraform plan` and checks. It runs commands on the two tagged instances through Systems Manager, and can create, change or delete nothing.

Evidence (TP-L03, TP-L04). The offline checker `scripts/verify-deployment.ps1` passes 19 of 19 checks: format and validation, no open ingress, no forbidden resources, allowed instance types, pinned images, only Nginx published, and the three images build. Two live sessions passed and were destroyed the same day, after which no instance, disk, address or security group was left. In the first, every service came up, and from the JMeter host only port 80 answered; ports 22, 3000, 5432, 5672, 6379, 8080, 9090 and 15672 were closed. A JMeter smoke test of 44 requests passed with 5 orders completed (`tests/evidence/20261022T093449Z-ec2-deployment.log`). The second was the overload run in Table 10.7 (`tests/evidence/20261022T163505Z-ec2-overload.log`).

10.10 Verification Summary

Figure 10.4 summarises the runtime verifiers' latest results; the counts are read from the saved logs by the chart script, not typed in. Each verifier starts its own throwaway broker and Redis against a throwaway verification database, and all of them were run again on the final code after the review fixes. Each was checked, when it was built, with a negative control that must make it fail, and every fix in Section 10.11 has one too.

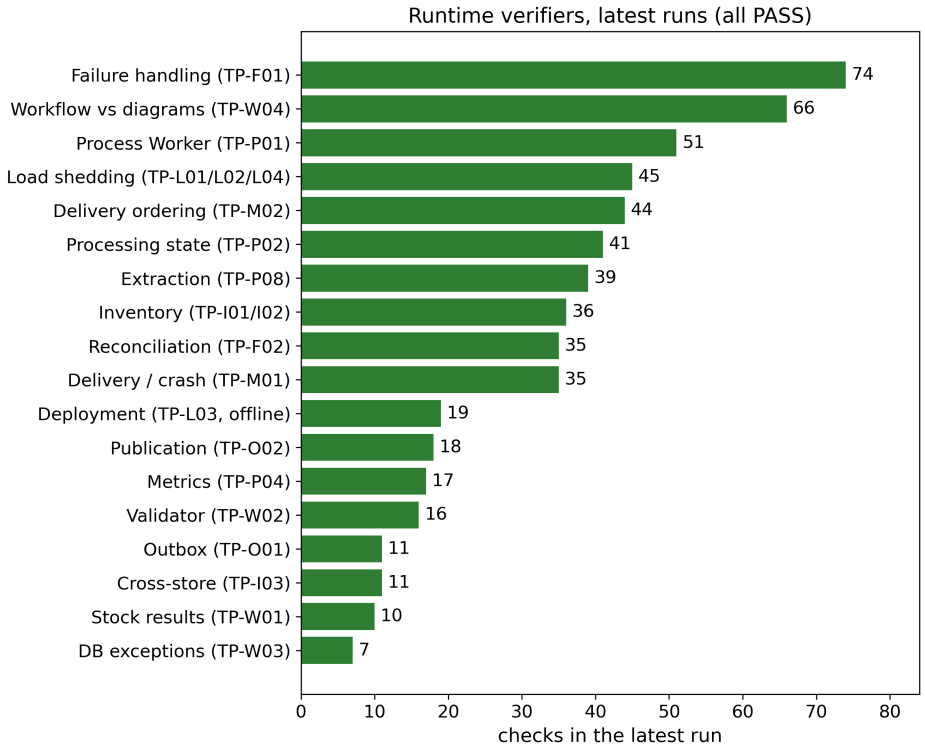


Figure 10.4. – Checks per runtime verifier in its latest run (no failures), plus 98 unit tests. Test-case IDs refer to `docs/test-plan.md`; each PASS links to a saved log in `tests/evidence/`.

10.11 External Review after Week 12

An external review graded Weeks 1–12 strictly. Every finding below was confirmed in the code or the saved data before it was fixed, and each fix has a test that fails when the fix is removed.

- **State regression.** The start and completion consumers use separate database contexts, so a late "processing started" event could write `Processing` over a `Completed` order. The order's state is now an optimistic-concurrency token: the late write matches no row, and the event is decided again from a fresh read and acknowledged as stale. A unit test runs the completion between the start event's read and its save, both with Entity Framework's exception and with the one ABP raises.
- **Hidden backlog and the in-flight bound.** No consumer limited its unacknowledged messages, so the queue bound did not cover the Worker's backlog, and the backlog measure missed it. Every consumer now has a prefetch of 20 (peak unacknowledged 252 before, 20 after), the measure counts all queued work, and the in-flight bound keeps the backlog small (Table 10.7).

- **Measurement.** The extractor now selects a run's orders by their own Idempotency-Keys (773, where the time window found 771), keeps one interval row per order when reconciliation re-publishes, stops on any SQL error instead of reading it as 0, and counts recovery only in windows without refusals and with accepted throughput. Publish times are taken when a message is sent, which removed the negative publish-to-decision intervals. Negative controls ran the previous extractor on the same inputs (`tests/evidence/20261023T123738Z-extract-negative-controls.log`).

- **Reconciliation** also recovers orders stuck in `Confirmed` or `Processing` (Section 10.6.3; the failure model is in Section 9.9).

- **Wording.** Order submissions are reported apart from status polls; the report no longer says a retry is never refused (Nginx can answer 429 first), that every consumer has an Inbox (the stateless Worker relies on Order Service's), or that the validator cannot report a false pass.

Still open: the EC2 overload run describes the code from before the review, and every limit is a pre-pilot value that the Week 13 pilot fixes.

11

Experimental Design

12 Results

This chapter is a set of empty templates, built in Week 11 and filled in Week 14 from the official runs. Each table and figure follows the teacher's evaluation plan (§6 of the brief) and the experiment plan: configurations C0–C4, the listed metrics, the six correctness checks, and 3–5 runs per configuration. Every cell is "—" until a run fills it. A dash cannot be mistaken for a measured zero.

12.1 Measurement Method

The configurations are those of the brief:

- **C0**, a naive read-then-write. It only shows the race condition and is not a baseline.
- **C1**, a synchronous PostgreSQL conditional atomic update. This is the main baseline.
- **C2**, the asynchronous pipeline: Redis Lua, RabbitMQ, Outbox, Inbox and retry.
- **C3**, C2 with rate limiting, concurrency limiting and bounded queues.
- **C4**, C3 with 1, 2 and 4 Inventory consumers on the same EC2 host.

Every run uses the same JMeter plan, run from a separate machine, and the same seeded stock (`scripts/reset-and-seed.ps1`, `scripts/warm-up.ps1`). After each run, `scripts/validate-correctness.ps1` gives its correctness verdict. A configuration that is fast but over-sells has not won anything, so no performance number is reported without that verdict.

Each configuration runs 3–5 times under the same conditions. For every run, latency is reported as p50, p95 and p99. A table cell gives the median of those values across runs, followed by the spread (range) across runs, for example "412 (398–431)". The number of runs is given in each caption.

Latency comes from two sources that are kept apart. JMeter measures what the client sees. The stage timestamps (Section 10.7) measure the pipeline inside the system. Where the two disagree, the disagreement is a result to report, not

an error to reconcile away. API throughput and business throughput are always reported separately, because an accepted request is not a completed order.

Each table and figure names the run ids its numbers come from (`experiments/RUN-ID/`). Unless a caption says otherwise, a table shows the steady-load, one-hot-product workload. The other workload profiles of the brief get the same tables in the appendix.

12.2 Correctness by Configuration

Table 12.1 comes first, because a performance number only counts for a configuration that passed these checks.

Table 12.1. – The six correctness checks of the brief, per configuration, from `validate-correctness.ps1` after every run. A cell gives the number of runs that passed out of the runs made. C0 is expected to fail: it exists to show the race. *To be completed in Week 14.*

Check	C0	C1	C2	C3	C4
Available inventory ≥ 0	—	—	—	—	—
Successful reservations $\leq$ initial inventory	—	—	—	—	—
At most one successful reservation per order	—	—	—	—	—
One downstream result per order	—	—	—	—	—
Every accepted order terminal or found by reconciliation	—	—	—	—	—
Orders by state add up to accepted requests	—	—	—	—	—
Units over-sold (worst run)	—	—	—	—	—

12.3 Latency by Configuration

Table 12.2 gives what the client sees. Table 12.3 compares C1 and C2–C4 at the same boundaries, and Table 12.4 splits the C2 pipeline into its stages.

Table 12.2. – Client-observed API latency (ms) of order submissions, from the JMeter JTL. Polling requests are reported separately in the appendix. Cells: median across runs (range). *To be completed in Week 14.*

Configuration	p50	p95	p99	Max	Runs
C1 – PostgreSQL atomic update	—	—	—	—	—
C2 – Redis event-driven	—	—	—	—	—
C3 – C2 + load shedding	—	—	—	—	—
C4 – C3, 1 Inventory consumer	—	—	—	—	—
C4 – C3, 2 Inventory consumers	—	—	—	—	—
C4 – C3, 4 Inventory consumers	—	—	—	—	—

Table 12.3. – Stock-decision latency and end-to-end completion latency (ms). Both start at the same request-start origin. The decision is C1's committed update, or Inventory's committed result for C2–C4. C1 has no Worker stage, so it has no completion latency. Cells: median across runs (range). *To be completed in Week 14.*

	Stock decision			Completion		
Configuration	p50	p95	p99	p50	p95	p99
C1	—	—	—	n/a	n/a	n/a
C2	—	—	—	—	—	—
C3	—	—	—	—	—	—
C4, 1 consumer	—	—	—	—	—	—
C4, 2 consumers	—	—	—	—	—	—
C4, 4 consumers	—	—	—	—	—	—

Table 12.4. – Time spent in each stage of the C2 pipeline (ms), from the stage timestamps (Section 10.7). Publication is marked after the broker's confirm, so a publish-to-decision interval can be slightly negative. Such intervals are counted and reported, never clamped to zero. *To be completed in Week 14.*

Stage	p50	p95	p99	Negative intervals
Accepted → event published	—	—	—	—
Event published → stock decided	—	—	—	—
Stock decided → decision applied	—	—	—	—
Decision applied → completed	—	—	—	—
Accepted → completed (total)	—	—	—	—

12.4 Throughput, Rejection and Errors

Table 12.5 keeps API throughput (accepted) apart from business throughput (completed), and stock rejection apart from overload rejection.

Table 12.5. – Throughput (orders per second) and rates (%), per configuration. Incoming is what JMeter offered. Accepted counts unique accepted orders, and an idempotent retry is not a new order. Completed counts unique orders that reached `Completed`. Reservation success and stock rejection share one denominator: unique stock decisions. Overload rejection (429/503) is not stock rejection. Cells: median across runs (range). *To be completed in Week 14.*

	Throughput (/s)			Rate (%)			
Config.	Incoming	Accepted	Completed	Reserved	Stock rej.	Overload rej.	Errors
C1	—	—	n/a	—	—	—	—
C2	—	—	—	—	—	—	—
C3	—	—	—	—	—	—	—
C4, 1 cons.	—	—	—	—	—	—	—
C4, 2 cons.	—	—	—	—	—	—	—
C4, 4 cons.	—	—	—	—	—	—	—

12.5 Overload, Backlog and Recovery

Tables 12.6 and 12.7 show what happens when input exceeds capacity, and how the reliability mechanisms behaved while it did.

Table 12.6. – Behaviour under the sustained-overload profile, followed by its recovery period. Queue depth and oldest message age are the peaks over the run, for the Inventory input queue. Oldest is the age of the oldest queued message. Pending counts orders not yet terminal when the run ends. Drain time runs from the end of submissions until every queue, the Outbox and the pending orders are empty. Recovery time runs from overload removal until latency, errors and throughput are back in the range set by the pilot. A value that could not be measured is marked unavailable, never 0. *To be completed in Week 14.*

Config.	Rejected (%)	Peak queue	Oldest (s)	Drain (s)	Recovery (s)	Pending
C2	—	—	—	—	—	—
C3	—	—	—	—	—	—
C4, 1 cons.	—	—	—	—	—	—
C4, 2 cons.	—	—	—	—	—	—
C4, 4 cons.	—	—	—	—	—	—

Table 12.7. – What the reliability mechanisms did, as totals per run (median across runs). Retries and duplicates are attempts, not business effects. A high count is a result to report, not a defect to hide. *To be completed in Week 14.*

Config.	Retries	Duplicates	Dead-lettered	Reconciliation	Outbox peak
C2	—	—	—	—	—
C3	—	—	—	—	—
C4, 1 cons.	—	—	—	—	—
C4, 2 cons.	—	—	—	—	—
C4, 4 cons.	—	—	—	—	—

12.6 Resource Use

Table 12.8 shows where each configuration spends the host, which is where a bottleneck shows up.

Table 12.8. – Resource use on the backend EC2 host during the steady-load profile: mean (peak). Collection settings are the same for every configuration. The broker column gives RabbitMQ's publish, delivery and acknowledgement rates in messages per second. *To be completed in Week 14.*

Config.	CPU (%)	RAM (GB)	DB conns	Redis (ms)	Publish / deliver / ack
C1	—	—	—	n/a	n/a
C2	—	—	—	—	— / — / —
C3	—	—	—	—	— / — / —
C4, 1 cons.	—	—	—	—	— / — / —
C4, 2 cons.	—	—	—	—	— / — / —
C4, 4 cons.	—	—	—	—	— / — / —

12.7 Figures

The figures show what a table cannot: behaviour over time around the overload and the recovery, the spread between runs, and the replica comparison.

Placeholder, to be generated in Week 14.

Figure 12.1. – Throughput over time for one run of C2 and one run of C3 under the ramp, spike, sustained-overload and recovery profile: incoming, accepted and completed orders per second. Source: the JMeter JTL, and `rate(flashsale_orders_accepted_total)` and `rate(flashsale_orders_outcome_total{outcome="Completed"})` scraped every 5 s.

Placeholder, to be generated in Week 14.

Figure 12.2. – Queue depth, Outbox backlog and pending orders over time, for the same runs as Figure 12.1. The drain and recovery times of Table 12.6 are marked on the time axis.

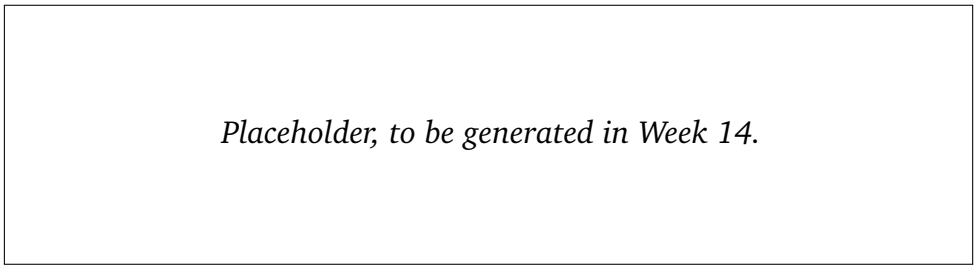


Figure 12.3. – API latency p50, p95 and p99 per configuration. Each run is a point and the median across runs is a bar, so the spread between runs stays visible.

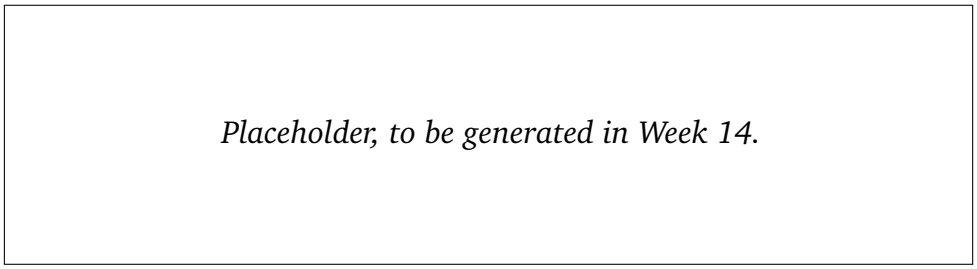


Figure 12.4. – C4 with 1, 2 and 4 Inventory consumers on one host: completed throughput, drain time and host CPU. All replicas share one machine, so this shows contention on that host, not multi-node scaling.

13

Discussion

14

Conclusion

References

A

Appendices